

**Design and Simulation of a Series-Parallel Hybrid Electric Vehicle (HEV) Using MATLAB
Simulink and Stateflow**

Project Documentation Report

KPIT Apex Lab

COEP Technological University, Pune, India

Submitted by

Raj Sawala MIS: 612407149 Email: sawalar24.extc@coeptech.ac.in Branch: ExTC

Amaan Mulla MIS: 612409027 Email: nasarma24.instru@coeptech.ac.in Branch: INSTRU

Karan Patil MIS: 612409031 Email: dilippk24.instru@coeptech.ac.in Branch: INSTRU

Mentor: Padma Dwivedi

Department of Electronics and Telecommunication Engineering

Academic Year 2025-26

ACKNOWLEDGEMENT

We sincerely express our gratitude to KPIT Apex Lab and COEP Technological University for providing the opportunity to work on this Hybrid Electric Vehicle modelling project. We thank our mentor, Padma Dwivedi, for guidance, reviews, and technical suggestions during the development and documentation of the model.

We also acknowledge the support received from faculty members, lab coordinators, and teammates during the modelling, debugging, and documentation phases. This project helped us understand HEV architecture, Simulink modelling, Stateflow-based control, Simscape physical plant integration, and scope-based validation of dynamic systems.

- Raj Sawala, Amaan Mulla, and Karan Patil

Abstract

This project presents a MATLAB Simulink and Stateflow model of a series-parallel Hybrid Electric Vehicle (HEV). The model combines a pack-level battery equivalent-circuit model, generator control, motor control, a Simscape physical plant, and an energy-management control subsystem. The control subsystem selects drive cycles, computes driver torque request from vehicle-speed feedback, chooses operating mode through a Stateflow chart, and allocates engine, motor, and generator torque through a dynamic torque-split MATLAB Function.

The physical plant contains Simscape drivetrain elements including ideal torque sources, inertias, gears, a disc friction clutch, Magic Formula tire, vehicle body, brake actuation, and engine-speed sensing. The battery subsystem uses a discrete-time pack-level 1RC equivalent-circuit update using dt , SOC_new , and V_RC_new feedback through Unit Delay blocks. The motor control subsystem applies speed-based torque limiting, SOC derating, regenerative braking protection, and slew-rate limiting. Generator control translates generator torque and engine-speed feedback into generator power/current using the efficiency input η with current clamping.

The documentation is intentionally written as a maintainable engineering report: each subsystem is described by purpose, interfaces, design logic, extracted code, parameters, expected behavior, scope outputs, assumptions, and known limitations. This makes the model easier to understand, review, modify, and extend.

Keywords
Hybrid Electric Vehicle, HEV, Series-Parallel Powertrain, Simulink, Stateflow, Simscape, Battery ECM, Energy Management System, Torque Split, Regenerative Braking

TABLE OF CONTENTS

Section	Page
Acknowledgement	2
Abstract	3
Nomenclature	5
Chapter 1: Introduction	6
Chapter 2: System Architecture	7
Chapter 3: Subsystem Interfaces	9
Chapter 4: Battery Model	12
Chapter 5: Generator Control	15
Chapter 6: Motor Control	17
Chapter 7: Physical Plant	18
Chapter 8: Energy Management Control	22
Chapter 9: Scopes and Result Documentation	32
Chapter 10: Parameter Reference	37
Chapter 11: Limitations and Assumptions	41
Chapter 12: Verification and Future Work	42
References	44
Appendix A: Extracted MATLAB Function Code	45
Appendix B: Model Inventory Tables	51
Appendix C: Parameter Initialization Script	56
Appendix D: Screenshot Index from Uploaded PDF	60

Note: Figure captions are placed below the corresponding screenshots and simulation results throughout the report.

Nomenclature

Abbreviation	Meaning
HEV	Hybrid Electric Vehicle
EMS	Energy Management System
ECM	Equivalent Circuit Model
SOC	State of Charge
OCV	Open Circuit Voltage
RC	Resistor-Capacitor network
PI	Proportional-Integral controller
P-only	Proportional-only controller
LUT	Look-Up Table
ICE	Internal Combustion Engine
PS	Physical Signal
RPM	Revolutions per minute
Nm	Newton metre

CHAPTER 1: INTRODUCTION

The objective of this project is to build and simulate a series-parallel HEV architecture. The model is not only a static block diagram; it represents closed-loop interaction between driver demand, energy-management decisions, battery state, generator charging, motor torque limiting, and the Simscape physical driveline.

The model is split into five major top-level subsystems: Battery, Generator_control, Motor_control, Physical Plant, and control. These subsystems exchange torque, speed, SOC, voltage, current, grade, clutch command, and braking signals. The most important documentation requirement is therefore not only to describe each block, but to describe the exact signal interfaces and how each subsystem affects the others.

Important documentation choice

The report records actual model details visible in the SLX package. Where a parameter is only a workspace symbol, such as C_nom, R1, C1, I_max_charge, T_gen_max, or dt, it is marked as an external workspace parameter rather than assigned an invented value.

1.1 Project objectives

- Model a series-parallel HEV powertrain with engine, motor, generator, clutch, gears, wheels, tire, and vehicle body.
- Implement a Stateflow-based EMS to switch between standby, regenerative braking, series, parallel, and series-parallel operating modes.
- Compute torque demand from vehicle-speed tracking and road-grade scenario selection.
- Represent the battery using a pack-level 1RC equivalent-circuit model with SOC dynamics and terminal-voltage calculation.
- Protect the motor through SOC derating, regeneration blocking at high SOC, speed-based torque limits, and slew-rate limiting.
- Provide scopes for torque, speed, power, battery, and generator/engine monitoring.

1.2 Report coverage

The report covers model architecture, exact subsystem ports, subsystem design notes, MATLAB Function code, Stateflow EMS transitions, parameter tables, scope documentation, limitations, and suggested verification checks. Screenshots from the provided PDF are inserted where available, and placeholders are included for additional screenshots and scope graphs that can be pasted later.

CHAPTER 2: SYSTEM ARCHITECTURE

At the top level, the model uses the control subsystem as the supervisory controller. In the current SLX file it receives actual vehicle speed, battery SOC, and engine speed; the drive-cycle selection is handled inside the control subsystem using the `ds_select` constant/manual selection path and the `dc_ts{1}...dc_ts{6}` workspace signals. It outputs torque commands and actuator commands to the physical plant, motor control, generator control, and scopes. The Battery subsystem provides SOC and voltage feedback. Motor_control and Generator_control act as actuator-level conditioning layers between control commands and electrical/powertrain behavior.

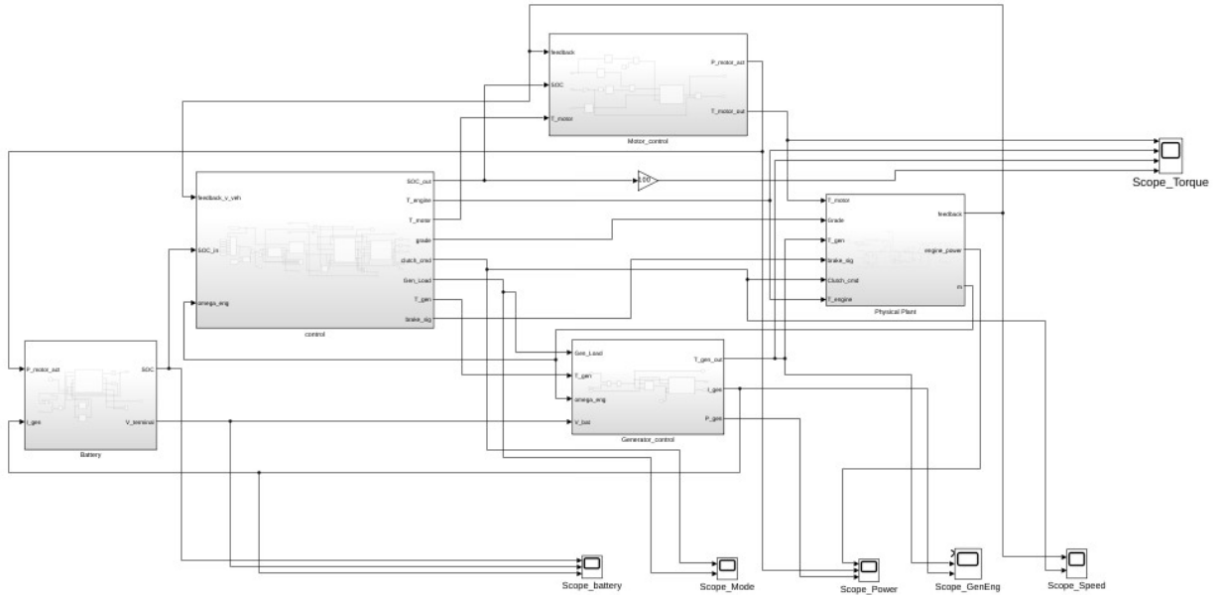


Figure 2.1 Top-level HEV model architecture from the uploaded screenshot PDF.

2.1 Top-level subsystem roles

Top-level subsystem	Role in the model
Battery	Pack-level battery equivalent-circuit subsystem. It estimates SOC and terminal voltage from motor power and generator charging current.
Generator_control	Converts generator torque/loading command and engine speed into actual generator torque, power, and charging current.
Motor_control	Applies speed, SOC, regenerative braking, and slew-rate limits to motor torque; computes actual motor power.
Physical Plant	Simscape driveline and vehicle dynamics plant. It converts torque/brake/grade commands into vehicle-speed and engine-speed feedback.
control	Energy-management system. It selects drive cycle, computes driver torque demand, chooses HEV mode in Stateflow, and calculates torque split.

2.2 High-level signal flow

1. An internal drive-cycle selector uses `ds_select`/manual selection logic to select one of six workspace drive-cycle signals `dc_ts{k}`.
2. DriverPI compares selected reference speed with `feedback_v_veh` and generates `T_req` and `brake_sig`.
3. `generate_terrain_scenario` passes `T_req` and assigns road grade based on the internally selected `Terrain_Mode` value.

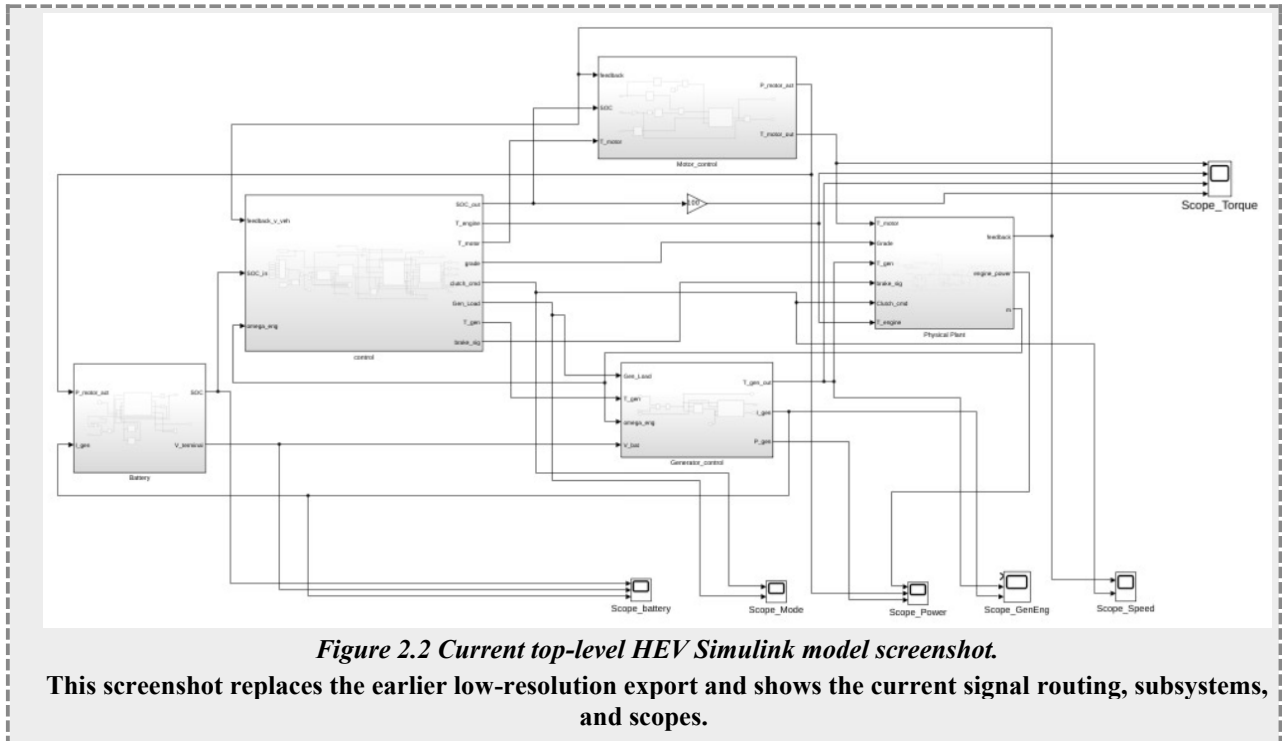
4.The Stateflow EMS chart uses vehicle speed, torque demand, SOC, grade, and high-torque threshold to decide the active HEV mode.

5.Torque_Split_Dynamic computes T_{engine} , T_{motor} , and T_{gen} according to EMS commands.

6.Motor_control limits the motor torque and computes P_{motor_act} for the Battery subsystem.

7.Generator_control saturates generator torque and computes I_{gen} for battery charging.

8.Physical Plant receives the torque/brake/grade commands and returns vehicle speed, engine speed, and engine power.



CHAPTER 3: SUBSYSTEM INTERFACES

The following tables list the actual subsystem ports observed in the SLX package. This section is important because HEV models become difficult to debug when torque, SOC, voltage, current, and command signals are mixed up.

3.1 control interface

Port	Input name	Unit / type	Description
1	feedback_v_veh	km/h	Actual vehicle speed feedback from Physical Plant.
2	SOC_in	%	Battery SOC signal after top-level 100x gain.
3	omega_eng	rpm	Engine speed feedback from Physical Plant.

Port	Output name	Unit / type	Description
1	SOC_out	%	Pass-through/monitoring SOC output.
2	T_engine	Nm	Engine torque command.
3	T_motor	Nm	Motor torque command before Motor_control limiting.
4	grade	%	Selected road grade.
5	clutch_cmd	0/1	Clutch command from EMS chart.
6	Gen_Load	0/1	Generator loading command from EMS chart.
7	T_gen	Nm	Generator torque command.
8	brake_sig	0-1	Brake signal from DriverPI.

3.2 Battery interface

Port	Input name	Unit / type	Description
1	P_motor_act	W (model signal)	Actual motor electrical power demand from Motor_control; divided by battery terminal voltage to estimate motor current.
2	I_gen	A	Generator charging current from Generator_control.

Port	Output name	Unit / type	Description
1	SOC	fraction, 0-1	Battery state of charge from the discrete BatteryECM update and Unit Delay feedback path. A 100x gain at top-level converts this to percent for control.
2	V_terminal	V	Pack terminal voltage after OCV, R0 and RC voltage drop scaling.

3.3 Generator_control interface

Port	Input name	Unit / type	Description
1	Gen_Load	0/1	EMS command enabling generator loading.
2	T_gen	Nm	Generator torque command from torque split calculator.
3	omega_eng	rpm (model signal)	Engine speed feedback from Physical Plant / engine sensor.

4	V_bat	V	Battery terminal voltage.
---	-------	---	---------------------------

Port	Output name	Unit / type	Description
1	T_gen_out	Nm	Actual/saturated generator torque sent to Physical Plant and scopes.
2	I_gen	A	Charging current sent to Battery and scopes.
3	P_gen	model power units	Generator power output computed in MATLAB Function.

3.4 Motor_control interface

Port	Input name	Unit / type	Description
1	feedback	km/h or speed feedback signal	Vehicle speed feedback from plant. A gain of 3.4722 generates omega_motor for motor power calculation.
2	SOC	%	SOC after top-level 100x gain.
3	T_motor	Nm	Motor torque command from control.

Port	Output name	Unit / type	Description
1	P_motor_act	W / model power units	Actual motor power computed from limited torque and speed.
2	T_motor_out	Nm	SOC-, speed- and slew-limited motor torque sent to plant and scope.

3.5 Physical Plant interface

Port	Input name	Unit / type	Description
1	T_motor	Nm	Motor torque applied through ideal torque source and motor gear.
2	Grade	%	Road grade command converted to road angle.
3	Clutch_cmd	0/1	Clutch engagement command.
4	T_gen	Nm	Generator loading torque.
5	brake_sig	0-1	Brake command from DriverPI.
6	T_engine	Nm	Engine torque command.

Port	Output name	Unit / type	Description
1	feedback	km/h	Vehicle speed feedback from Vehicle Body V output through PS-Simulink converter and 3.6 gain.
2	engine_power	W / model power units	Engine mechanical power from torque-speed sensor subsystem.
3	m	rpm / engine-speed feedback signal	Filtered engine-speed output from the engine sensor subsystem. It is routed at top level into omega_eng inputs of control and Generator_control.

Port-count note

In the live SLX package, Physical Plant has 6 inputs and 3 outputs: feedback, omega, and engine_power. If an older note says 6 inputs and 2 outputs, that appears outdated for this uploaded model.

CHAPTER 4: BATTERY MODEL

The Battery subsystem represents the energy source/sink of the HEV model. It receives actual motor power demand and generator charging current, estimates the net battery current, integrates SOC, and computes terminal voltage using a pack-level 1RC equivalent-circuit model.

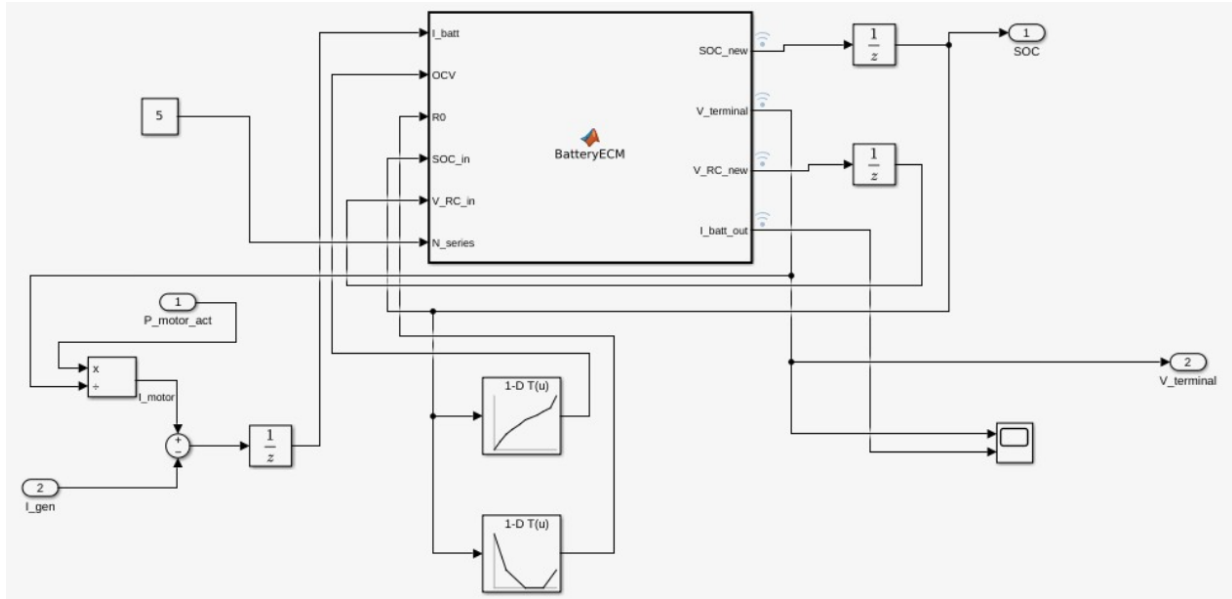


Figure 4.1 Battery subsystem screenshot.

4.1 Battery design overview

The battery model is based on a 1RC equivalent circuit. The open-circuit voltage and ohmic resistance are obtained from SOC-based lookup tables. In the current SLX file, the MATLAB Function performs a discrete-time SOC and RC-voltage update using the dt input. SOC_new and V_RC_new are fed back through Unit Delay blocks, so the function directly computes the next states rather than outputting continuous derivatives.

Battery block / parameter	Value visible in model	Purpose
SOC memory / Unit Delay initial condition	SOC_init (workspace symbol)	Initial SOC is provided from workspace. In the included Drive Cycle 2 result, SOC starts near 90%.
RC voltage memory / Unit Delay initial condition	V_RC_init (workspace symbol)	Initial RC polarization voltage used by the discrete 1RC battery update.
N_series constant	5	Scales cell-level OCV/R0/V_RC to pack level.
OCV lookup	SOC -> cell voltage	Provides cell open-circuit voltage curve.
R0 lookup	SOC -> ohmic resistance	Provides SOC-dependent internal resistance.
Unit Delay3 sample time	0.01 s	Breaks direct feedthrough/current path and supports simulation stability.
BatteryECM time step	dt input	Used for Coulomb counting and RC exponential update.

4.2 Battery mathematical formulation

The implemented equations are:

$$dSOC = -I_{batt} * dt / (C_{nom} * 3600)$$

$$SOC_{new} = \max(0, \min(1, SOC_{in} + dSOC))$$

$$\alpha = \exp(-dt/(R1*C1))$$

$$V_{RC_new} = \alpha * V_{RC_in} + R1 * (1 - \alpha) * I_{batt}$$

$$OCV_{pack} = OCV * N_{series}$$

$$R0_{pack_drop} = I_{batt} * R0 * N_{series}$$

$$V_{RC_pack} = V_{RC_new} * N_{series}$$

$$V_{terminal} = OCV_{pack} - R0_{pack_drop} - V_{RC_pack}$$

$V_{terminal}$ is clamped between $2.5 * N_{series}$ and $4.3 * N_{series}$.

Pack scaling

The uploaded model passes $N_{series} = 5$ to the BatteryECM function. This makes the terminal-voltage clamp 12.5 V to 21.5 V. For a full-size HEV, this is a simplified low-voltage pack representation unless the workspace/model is later re-parameterized.

4.3 Battery MATLAB Function code

```
function [dSOC, V_terminal, dV_RC, I_batt_out] = ...
    BatteryECM(I_batt, OCV, R0, SOC_in, V_RC_in, C_nom, R1, C1, N_series)

%% 1. SOC Derivative (Rate of discharge)
dSOC = -I_batt / (C_nom * 3600);

%% 2. RC Continuous Differential Equation
% dVc/dt = -Vc/(R1*C1) + I/(C1)
dV_RC = -V_RC_in / (R1*C1) + I_batt / C1;

%% 3. Terminal Voltage (Pack Level)
OCV_pack = OCV * N_series;
R0_pack_drop = I_batt * R0 * N_series;
V_RC_pack = V_RC_in * N_series; % Using the current state, not a predicted state

V_terminal = OCV_pack - R0_pack_drop - V_RC_pack;
V_terminal = max(2.5*N_series, min(4.3*N_series, V_terminal)); % pack-level clamp

%% 4. Pass-through
I_batt_out = I_batt;
end
```

Figure 4.2 BatteryECM code screenshot from PDF.

Listing 4.1 BatteryECM MATLAB Function extracted from the SLX package.

```
function [SOC_new, V_terminal, V_RC_new, I_batt_out] = ...
    BatteryECM(I_batt, OCV, R0, SOC_in, V_RC_in, dt, C_nom, R1, C1, N_series)

%% 1. SOC integration (Coulomb counting)
dSOC = -I_batt * dt / (C_nom * 3600);
SOC_new = SOC_in + dSOC;
SOC_new = max(0, min(1, SOC_new));

%% 2. RC pair dynamics at cell level
alpha = exp(-dt/(R1*C1));
```

```
V_RC_new = alpha*V_RC_in + Rl*(1-alpha)*I_batt;  
  
%% 3. Terminal voltage at pack level  
OCV_pack    = OCV * N_series;  
R0_pack_drop = I_batt * R0 * N_series;  
V_RC_pack    = V_RC_new * N_series;  
  
V_terminal = OCV_pack - R0_pack_drop - V_RC_pack;  
V_terminal = max(2.5*N_series, min(4.3*N_series, V_terminal));  
  
%% 4. Pass-through  
I_batt_out = I_batt;  
  
end
```

CHAPTER 5: GENERATOR CONTROL

Generator_control acts as the conditioning layer between the EMS generator torque command and the battery charging current. It multiplies the generator command by Gen_Load, applies a saturation block, and computes generator power/current through a MATLAB Function.

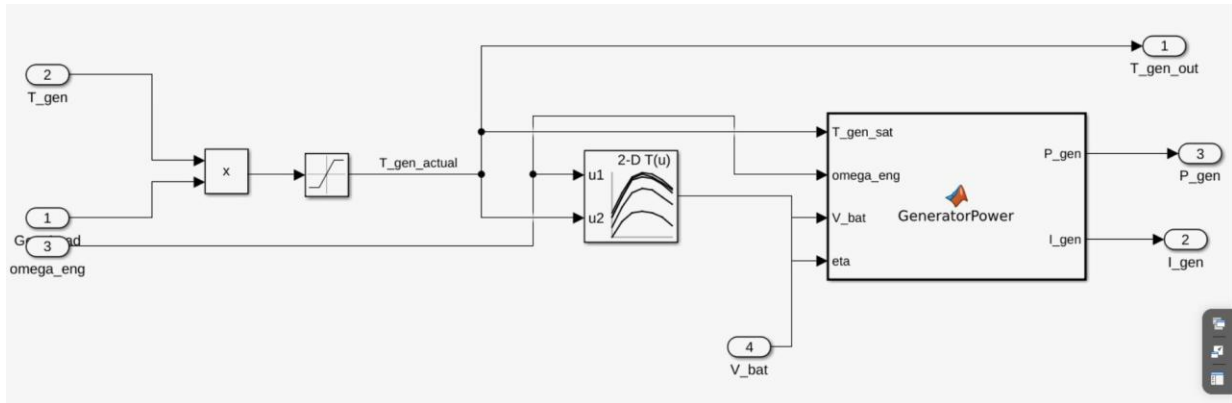


Figure 5.1 Generator_control subsystem screenshot.

5.1 Generator control signal path

- Gen_Load and T_gen are multiplied so that generator torque is active only when the EMS enables generator loading.
- The product is saturated using lower limit 0 and upper limit T_gen_max.
- T_gen_out is sent back to the plant and to scopes as actual/saturated generator torque.
- GeneratorPower receives T_gen_sat, omega_eng, V_bat, and eta, then outputs P_gen and I_gen.
- I_gen is clamped to I_max_charge and constrained to be non-negative.

5.2 Generator lookup table and implementation note

Item	Extracted value	Engineering meaning
2-D LUT breakpoints dimension 1	[0;50;100;150;200;250;300]	Speed/torque-map axis as implemented. Connected to omega_eng in the subsystem.
2-D LUT breakpoints dimension 2	[0;20;40;60;80]	Second map axis. Connected to T_gen_actual in the subsystem.
2-D LUT table	7 x 5 matrix, values approx. 0.60 to 0.92	Represents an efficiency-like map.
Actual connection status	LUT output is connected to the eta input of GeneratorPower.	The current GeneratorPower function multiplies generator torque and engine speed by eta before current calculation.

Implementation update note
The current Generator_control screenshot and SLX function signature show eta connected to GeneratorPower. The current GeneratorPower code computes $P_{gen} = T_{gen_sat} * \omega_{eng} * \eta$, then divides by safe battery voltage and clamps I_gen to I_max_charge.

5.3 Generator MATLAB Function code

```
function [P_gen,I_gen] = GeneratorPower(T_gen_sat, omega_eng, V_bat, I_max_charge)

% Electrical power generation
P_gen = T_gen_sat * omega_eng;

% Avoid divide-by-zero
V_nom_min = 1;
V_safe = max(V_bat, V_nom_min);

% Charging current
I_gen = P_gen / V_safe;

% Current limit
I_gen = min(I_gen, I_max_charge);

% No negative charging current
I_gen = max(I_gen,0);

end
```

Figure 5.2 GeneratorPower code screenshot from PDF.

Listing 5.1 GeneratorPower MATLAB Function extracted from the SLX package.

```
function [P_gen,I_gen] = GeneratorPower(T_gen_sat, omega_eng, V_bat, eta, I_max_charge)

% Electrical power generation
P_gen = T_gen_sat * omega_eng * eta;

% Avoid divide-by-zero
V_nom_min = 1;
V_safe = max(V_bat, V_nom_min);

% Charging current
I_gen = P_gen / V_safe;

% Current limit
I_gen = min(I_gen, I_max_charge);

% No negative charging current
I_gen = max(I_gen,0);

end
```


CHAPTER 6: MOTOR CONTROL

Motor_control receives the torque command from the EMS and applies practical limits before sending motor torque to the physical plant. It also estimates actual motor power for the battery model.

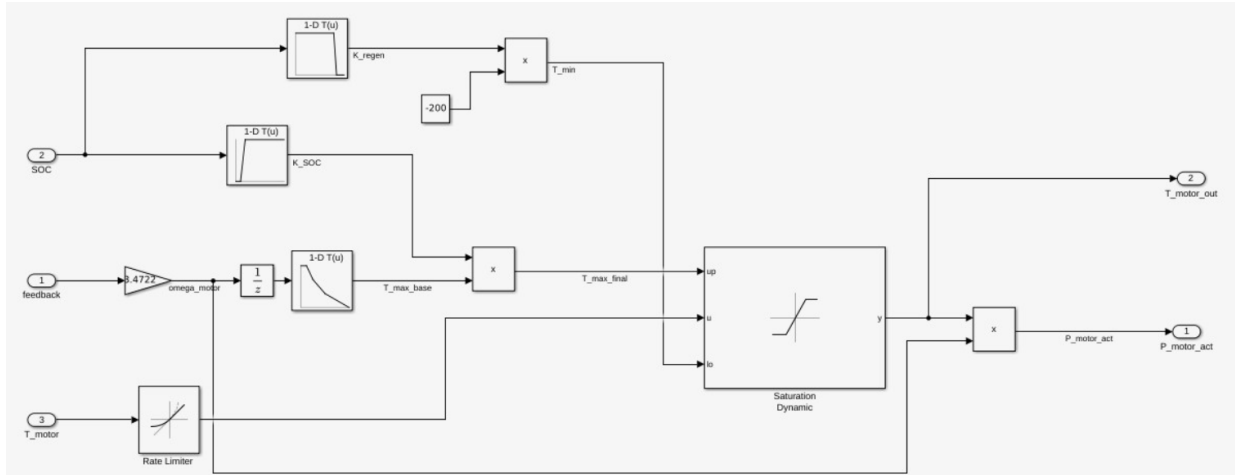


Figure 6.1 Motor_control subsystem screenshot.

6.1 Motor-control protective behavior

- Speed-based torque limiting reduces maximum motor torque as motor speed increases.
- K_SOC derates positive motor torque at low SOC. In the uploaded table, motor assistance is zero below 10% SOC and reaches full permission by 20% SOC.
- K_regen derates regenerative braking at high SOC. Regeneration remains allowed up to 80% SOC and is blocked by 85% SOC.
- A dynamic saturation block limits the requested torque between the computed upper and lower torque limits.
- The Rate Limiter uses rising slew limit 500 and falling slew limit -500, with inherited sample time.

Motor parameter / LUT	Breakpoints	Table values	Purpose
T_max_base LUT	[0;50;100;200;400]	[250;250;200;150;100]	Speed-based base motor torque limit.
K_SOC LUT	[0;0.1;0.2;1]	[0;0;1;1]	Low-SOC motor-assist derating.
K_regen LUT	[0;0.8;0.85;1]	[1;1;0;0]	High-SOC regenerative braking blocking.
T_min constant	-200	-	Lower motor torque limit before K_regen scaling.
Feedback gain	3.4722	-	Generates omega_motor from feedback signal for motor power calculation.
Rate limiter	Rising 500, falling -500	-	Limits torque-command slew rate.

Motor-control subsystem screenshot has been inserted above. Lookup-table numeric breakpoints and table values are documented in the table; separate block-dialog screenshots may be added later if required by the evaluator.

CHAPTER 7: PHYSICAL PLANT

The Physical Plant subsystem is the Simscape driveline model. It converts engine, motor, generator, brake, clutch, and grade commands into vehicle motion and engine-speed feedback. It is the most physically detailed part of the model.

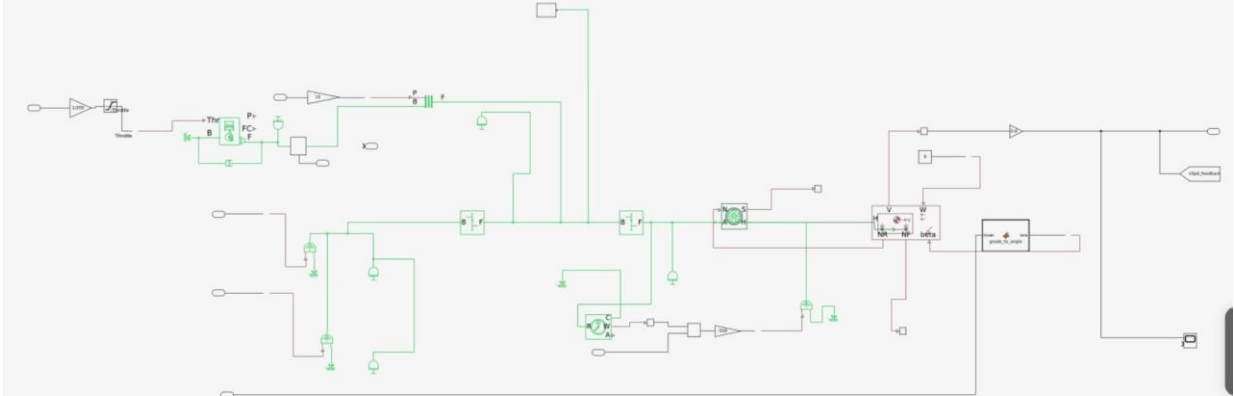


Figure 7.1 Physical Plant subsystem screenshot.

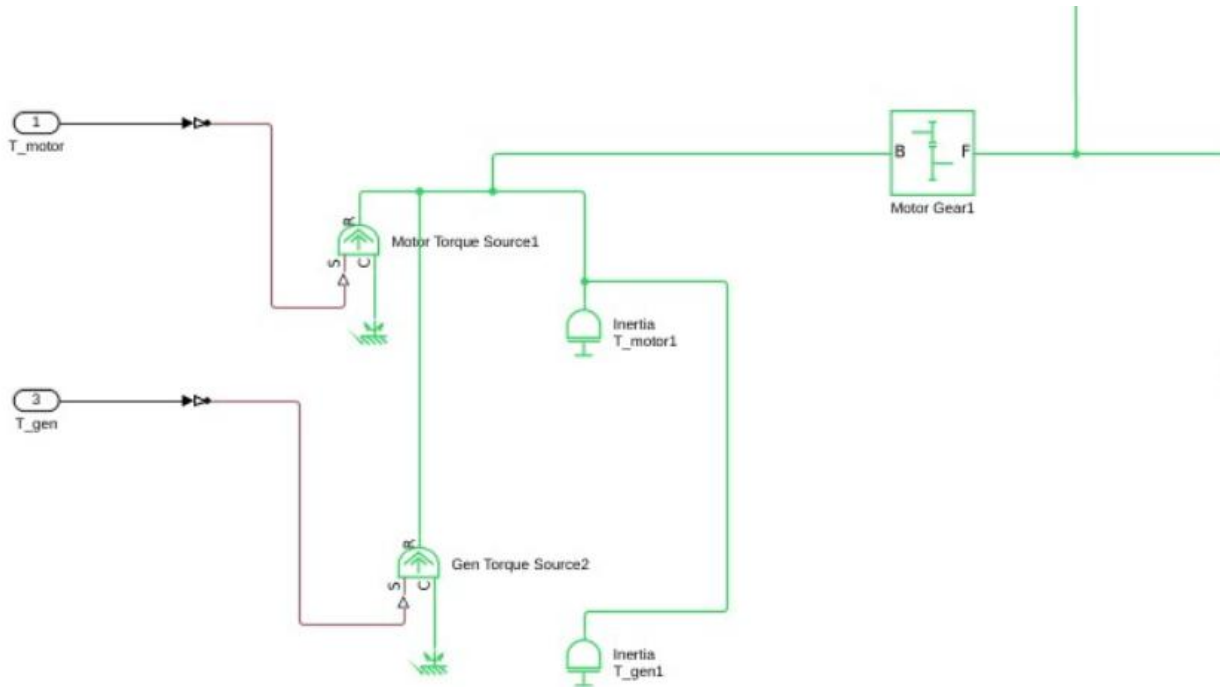


Figure 7.1(b) Physical Plant zoomed view of the motor/generator torque-source and motor-gear section.

7.1 Mechanical topology

The topology visible in the uploaded screenshot and SLX XML can be summarized as follows: engine torque is applied through an engine subsystem and engine shaft inertia; a clutch controls mechanical coupling to the driveline; motor and generator torque sources are connected through their own inertias; the motor path uses a Motor Gear; the driveline reaches the Final Drive gear, wheel inertia, Magic Formula tire, and Vehicle Body. Vehicle speed and engine speed are sensed and returned to the controller.

Plant component	Block type / library source	Visible parameter values	Purpose
Generic Engine	Simscape Driveline Generic Engine	Initial velocity 800 rpm; redline speed 5000 rpm; inertia model off in block; engine	Engine-side mechanical source/behavior.

		shaft inertia added separately	
Engine Shaft Inertia	Rotational Inertia	$0.25 \text{ kg}\cdot\text{m}^2$	Dedicated shaft inertia for engine dynamics.
Disc Friction Clutch	Simscape Driveline Clutch	$r_{\text{eff}} = 130 \text{ mm}$; piston area = 0.001 m^2 ; $\mu_{\text{static}} = 0.35$	Engages/disengages engine mechanical path.
Motor Torque Source	Ideal Torque Source	Input from T_{motor}	Applies commanded motor torque.
Generator Torque Source	Ideal Torque Source	Input from T_{gen}	Applies generator loading torque.
Motor Gear1	Simple Gear	ratio = 2; efficiency = 0.8	Motor/driveline gearing.
Final Drive gear1	Simple Gear	ratio = 2; efficiency = 0.8	Final driveline gear reduction.
Tire (Magic Formula)1	Magic Formula Tire	$p_{\text{tire}} = 250\text{e}3 \text{ Pa}$; damping = $1000 \text{ N}\cdot\text{s}/\text{m}$	Tire-road longitudinal force.
Vehicle Body1	Vehicle Body	mass = 1200 kg ; area = 3 m^2 ; $C_d = 0.4$; $\rho = 1.18$	Vehicle longitudinal dynamics.

7.2 Engine/sensor subsystem design

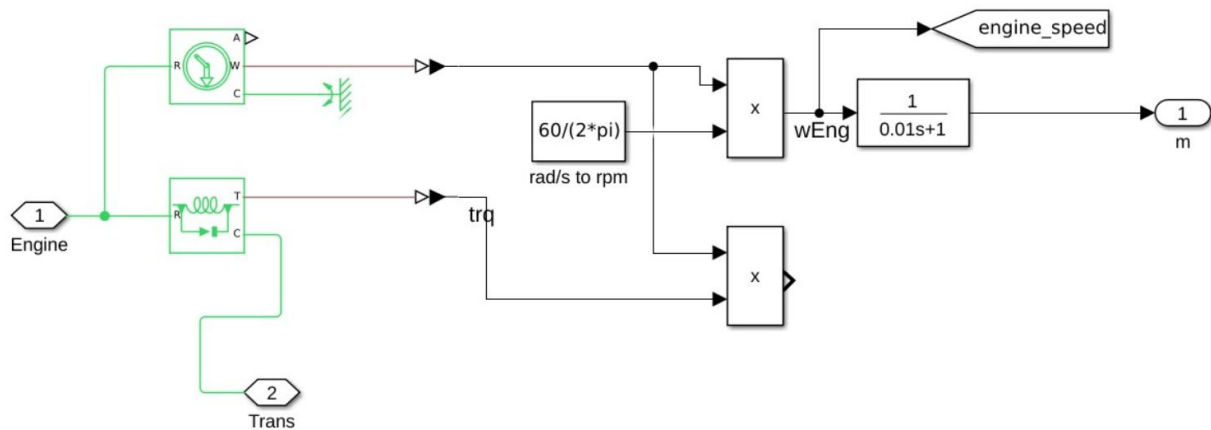


Figure 7.2 Engine sensor subsystem screenshot.

The engine-speed sensing subsystem uses an Ideal Rotational Motion Sensor and conversion from rad/s to rpm using $60/(2\pi)$. Engine torque and angular velocity are also multiplied to generate engine power. A first-order transfer function $1/(0.01s+1)$ is used on the engine-speed output path, giving a filtered ω_{engine} signal for feedback.

Rationale for this plant style

The model uses a Simulink-computed torque command with ideal torque sources and dedicated inertias. This is appropriate for a student-level HEV control project because it keeps the EMS and torque split logic visible and tunable, instead of hiding engine behavior inside a combustion-calibrated block requiring detailed engine maps.

7.3 Grade conversion code

```
function beta = grade_to_angle(Grade)
%#codegen
beta = atan(Grade/100);
end
```

Figure 7.3 grade_to_angle code screenshot from PDF.

Listing 7.1 *grade_to_angle* MATLAB Function extracted from the SLX package.

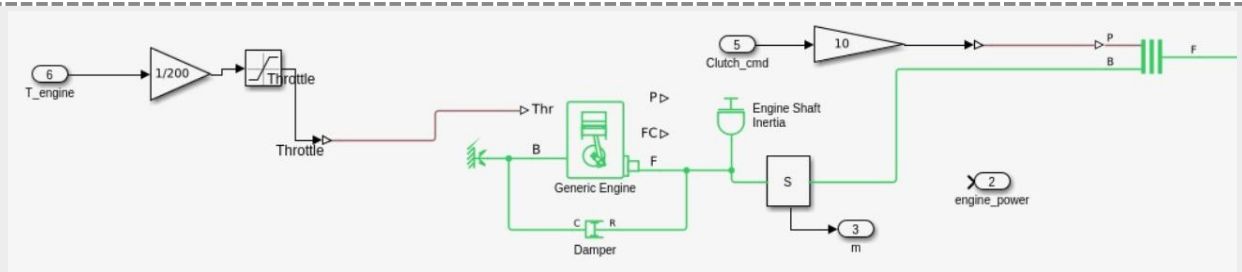
```
function beta = grade_to_angle(Grade)

%#codegen

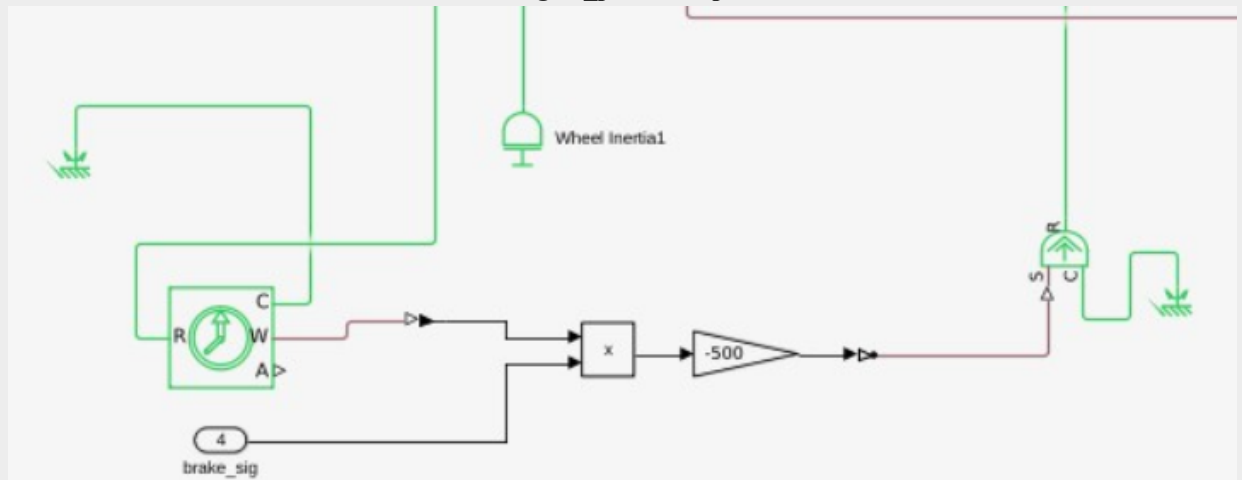
beta = atan(Grade/100);

end
```

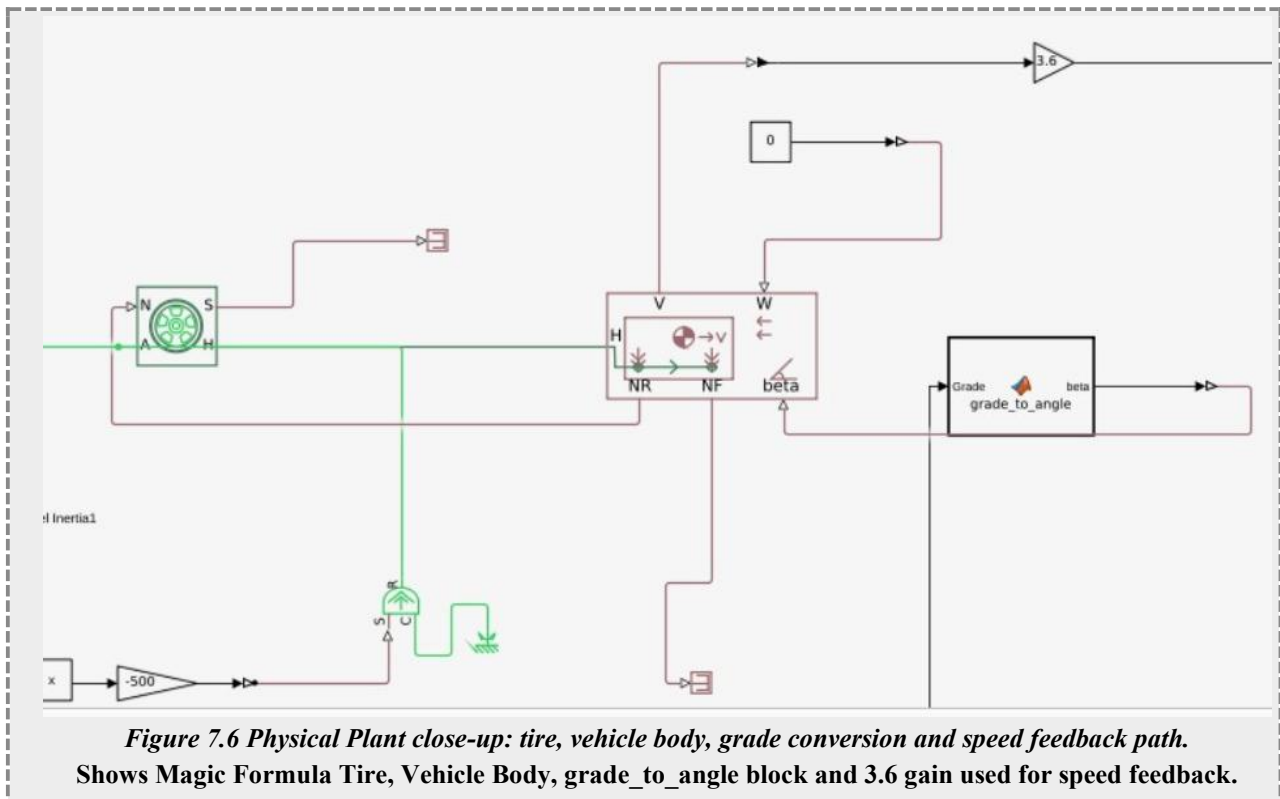
Additional physical-plant close-up screenshots inserted below: engine/clutch path, brake actuator path, and tire/vehicle-body/grade path.

**Figure 7.4** *Physical Plant close-up: engine, throttle, clutch command and engine shaft path.*

Shows the torque command input, throttle conversion, Generic Engine, shaft inertia, clutch command path and engine_power output.

**Figure 7.5** *Physical Plant close-up: brake actuator and wheel-side braking path.*

Shows brake_sig path, braking torque conversion and connection to the wheel/driveline side.



CHAPTER 8: ENERGY MANAGEMENT CONTROL

The control subsystem acts as the EMS. It selects the drive cycle, computes torque demand, assigns road grade, selects HEV operating mode using Stateflow, and calculates engine/motor/generator torque commands.

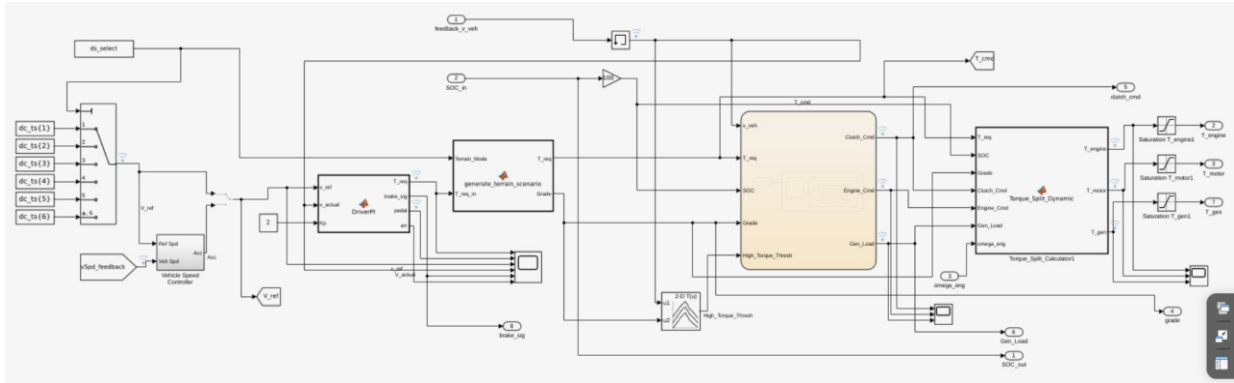


Figure 8.1 control subsystem screenshot.

8.1 Drive-cycle selection

The control subsystem contains six From Workspace drive-cycle signals, `dc_ts{1}` to `dc_ts{6}`, selected through the internal drive-cycle selection path. In the current SLX file the top-level control subsystem no longer exposes `Terrain_mode` as an external input port; selection is handled inside control using `ds_select`/manual selection logic. Several From Workspace blocks use the workspace sample time `dt`, so `dc_ts` and `dt` must be available before simulation.

Terrain_Mode	Drive-cycle source	Grade assigned by generate_terrain_scenario
1	<code>dc_ts{1}</code>	0% flat road
2	<code>dc_ts{2}</code>	1% slight road grade
3	<code>dc_ts{3}</code>	8% steep uphill road
4	<code>dc_ts{4}</code>	0% flat road
5	<code>dc_ts{5}</code>	1% slight road grade
6	<code>dc_ts{6}</code>	8% steep uphill road
otherwise	none	$T_{req}=0$, Grade=0 fail-safe

8.2 DriverPI current control law

In the current SLX file, `DriverPI` is now a true PI-style speed controller with proportional gain K_p , integral gain K_i , and time step dt . The function accumulates error using $int_err = int_err + err*dt$, applies anti-windup limiting when K_i is non-zero, saturates pedal command to $[-1,1]$, generates T_{req} from positive pedal demand, generates `brake_sig` from negative pedal demand, and smooths T_{req} using a low-pass filter with $\alpha = 0.02$.

```

function [T_req, brake_sig, pedal, err] = DriverPI(v_ref, v_actual, Kp)
err = v_ref - v_actual;
pedal = Kp * err;
pedal = max(-1, min(1, pedal));

persistent T_req_filt
if isempty(T_req_filt)
    T_req_filt = 0;
end
alpha = 0.02; % filter strength, 0=no filtering, 1=instant (no smoothing)
T_req_raw = max(0, pedal) * 280;
T_req_filt = T_req_filt + alpha*(T_req_raw - T_req_filt);
T_req = T_req_filt;

brake_sig = max(0, -pedal);
end

```

Figure 8.2 DriverPI code screenshot from PDF.

Listing 8.1 DriverPI MATLAB Function extracted from the SLX package.

```

function [T_req, brake_sig, pedal, err] = DriverPI(v_ref, v_actual, Kp, Ki, dt)

% 1. Initialize memory

persistent int_err T_req_filt

if isempty(int_err)

    int_err = 0;

    T_req_filt = 0;

end

% 2. Error calculation

err = v_ref - v_actual;

% 3. Proper time-based integration

int_err = int_err + (err * dt);

% 4. Anti-windup

if Ki ~= 0

    int_limit = 1 / abs(Ki);

    int_err = max(-int_limit, min(int_limit, int_err));

end

% 5. PI control law

pedal_unsat = (Kp * err) + (Ki * int_err);

pedal = max(-1, min(1, pedal_unsat));

% 6. Torque and braking commands

T_req_raw = max(0, pedal) * 280;

brake_sig = max(0, -pedal);

```

```
% 7. Low-pass filter for T_req
```

```
alpha = 0.02;
```

```
T_req_filt = T_req_filt + alpha * (T_req_raw - T_req_filt);
```

```
T_req = T_req_filt;
```

```
end
```

8.3 Terrain/grade scenario function

```
function [T_req, Grade] = generate_terrain_scenario(Terrain_Mode, T_req_in)
% #codegen
% This block generates simulated vehicle states based on selected terrain.
%
% Input:
%   Terrain_Mode - 1 = City, 2 = Highway, 3 = Mountain, etc.
%
% Outputs:
%   T_req - Driver torque request in Nm
%   Grade - Road gradient in %

switch Terrain_Mode
case 1
    % drive cycle 1
    T_req = T_req_in; % Nm
    Grade = 0.0; % flat road
case 2
    % drive cycle 2
    T_req = T_req_in; % Nm
    Grade = 1.0; % slight road grade
case 3
    % drive cycle 3
    T_req = T_req_in; % Nm
    Grade = 8.0; % steep uphill road
case 4
    % drive cycle 4
    T_req = T_req_in; % Nm
    Grade = 0.0; % flat road
case 5
    % drive cycle 5
    T_req = T_req_in; % Nm
    Grade = 1.0; % slight road grade
case 6
    % drive cycle 6
    T_req = T_req_in; % Nm
    Grade = 8.0; % steep uphill road
otherwise
    % DEFAULT / FAILSAFE
    T_req = 0.0; % Safely zero out torque demand
    Grade = 0.0; % Assume flat ground
end
end
```

Figure 8.3 generate_terrain_scenario code screenshot from PDF.

Listing 8.2 generate_terrain_scenario MATLAB Function extracted from the SLX package.

```
function [T_req, Grade] = generate_terrain_scenario(Terrain_Mode, T_req_in)

% #codegen

% This block generates simulated vehicle states based on selected terrain.

%

% Input:

%   Terrain_Mode - 1 = City, 2 = Highway, 3 = Mountain, etc.

%

% Outputs:
```



```

% T_req - Driver torque request in Nm

% Grade - Road gradient in %

switch Terrain_Mode

    case 1

        % drive cycle 1

        T_req = T_req_in; % Nm

        Grade = 0.0; % flat road

    case 2

        % drive cycle 2

        T_req = T_req_in; % Nm

        Grade = 1.0; % slight road grade

    case 3

        % drive cycle 3

        T_req = T_req_in; % Nm

        Grade = 8.0; % steep uphill road

    case 4

        % drive cycle 4

        T_req = T_req_in; % Nm

        Grade = 0.0; % flat road

    case 5

        % drive cycle 5

        T_req = T_req_in; % Nm

        Grade = 1.0; % slight road grade

    case 6

        % drive cycle 6

        T_req = T_req_in; % Nm

        Grade = 8.0; % steep uphill road

    otherwise

        % DEFAULT / FAILSAFE

        T_req = 0.0; % Safely zero out torque demand

        Grade = 0.0; % Assume flat ground

end

end

```

8.4 High torque threshold lookup

The control subsystem contains a 2-D lookup table for High_Torque_Thresh. The breakpoints and table values visible in the SLX XML are shown below. This threshold is used in the Stateflow EMS transition into series-parallel mode when grade and torque demand are high.

Axis / table	Extracted value
Breakpoints dimension 1	[0;20;40;60;70;80;100;120]

Breakpoints dimension 2	[0 5 10]
Table	[100 80 60; 120 100 80; 140 120 100; 155 135 115; 145 125 105; 130 110 90; 115 95 75; 95 80 65]

8.5 EMS Stateflow mode-selection chart

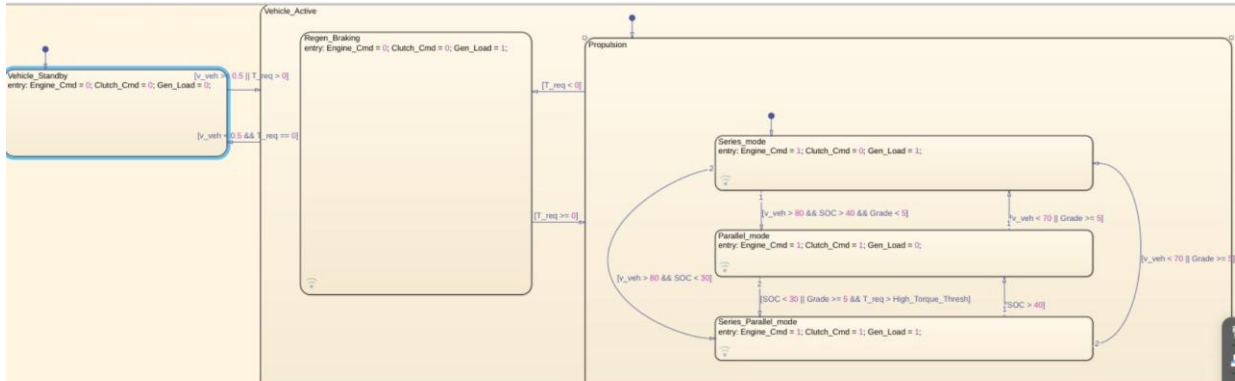


Figure 8.4 EMS Stateflow chart screenshot.

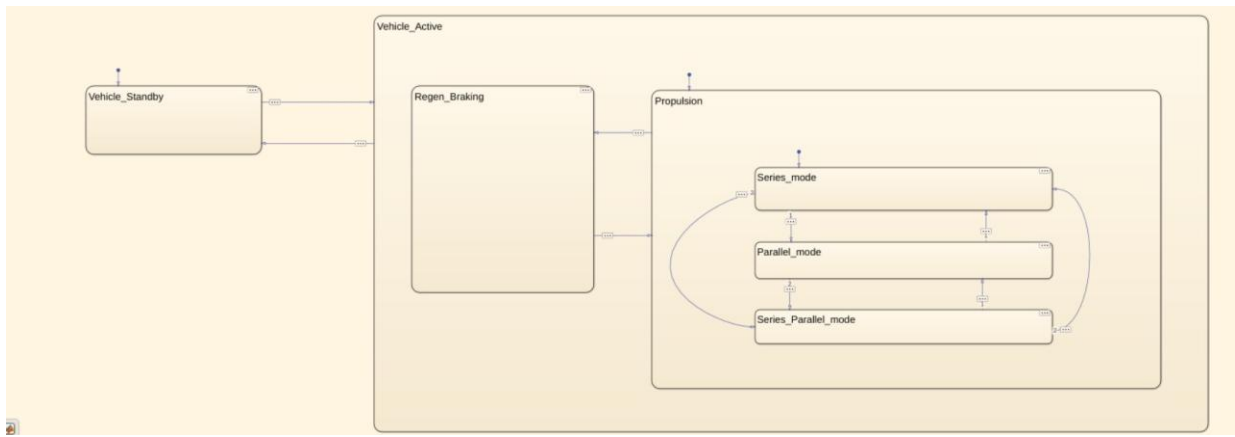


Figure 8.4(b) Alternate EMS Stateflow overview showing the same state hierarchy with a cleaner zoomed layout.

The EMS chart contains the top states `Vehicle_Standby` and `Vehicle_Active`. Inside `Vehicle_Active`, the model separates `Regen_Braking` and `Propulsion`. Inside `Propulsion`, three operating modes are implemented: `Series_mode`, `Parallel_mode`, and `Series_Parallel_mode`.

State ID	State	Entry action / description
51	Vehicle_Active	Vehicle_Active
57	Regen_Braking	Regen_Braking; entry: Engine_Cmd = 0; Clutch_Cmd = 0; Gen_Load = 1;
59	Propulsion	Propulsion
66	Series_mode	Series_mode; entry: Engine_Cmd = 1; Clutch_Cmd = 0; Gen_Load = 1;
67	Parallel_mode	Parallel_mode; entry: Engine_Cmd = 1; Clutch_Cmd = 1; Gen_Load = 0;
68	Series_Parallel_mode	Series_Parallel_mode; entry: Engine_Cmd = 1; Clutch_Cmd = 1; Gen_Load = 1;
50	Vehicle_Standby	Vehicle_Standby; entry: Engine_Cmd = 0; Clutch_Cmd = 0; Gen_Load = 0;

8.6 EMS transition conditions

Transition ID	Source	Destination	Condition
---------------	--------	-------------	-----------

124	Default/Initial	Series_mode	(default transition)
81	Series_Parallel_mode	Series_mode	[v_veh < 70 Grade >= 5]
76	Series_mode	Series_Parallel_mode	[v_veh > 80 && SOC < 30]
75	Series_mode	Parallel_mode	[v_veh > 80 && SOC > 40 && Grade < 5]
78	Parallel_mode	Series_mode	[v_veh < 70 Grade >= 5]
79	Parallel_mode	Series_Parallel_mode	[SOC < 30 Grade >= 5 && T_req > High_Torque_Thresh]
80	Series_Parallel_mode	Parallel_mode	[SOC > 40]
85	Default/Initial	Propulsion	(default transition)
87	Propulsion	Regen_Braking	[T_req < 0]
88	Regen_Braking	Propulsion	[T_req >= 0]
92	Default/Initial	Vehicle_Standby	(default transition)
53	Vehicle_Standby	Vehicle_Active	[v_veh >= 0.5 T_req > 0]
56	Vehicle_Active	Vehicle_Standby	[v_veh < 0.5 && T_req == 0]

Transition precedence note

The condition [SOC < 30 || Grade >= 5 && T_req > High_Torque_Thresh] follows MATLAB/Stateflow logical precedence, where && binds tighter than ||. In plain words: SOC below 30 triggers series-parallel, or high grade plus high torque triggers it.

8.7 Torque_Split_Dynamic logic

Torque_Split_Dynamic converts EMS commands into actual torque requests for engine, motor, and generator. It uses command combinations from the Stateflow chart. The function also includes vehicle mass, wheel radius, grade force calculation, SOC weighting, and final torque clamping.

EMS command combination	Mode interpreted by code	Torque allocation behavior
Engine_Cmd=0, Clutch_Cmd=0, Gen_Load=1	Standby or regenerative braking	If T_req<0, motor torque is negative and scaled by eta_motor; otherwise motor torque is zero.
Engine_Cmd=1, Clutch_Cmd=0, Gen_Load=1	Series mode	Motor provides propulsion torque up to T_motor_max. Engine runs near T_engine_opt plus SOC correction and drives generator.
Engine_Cmd=1, Clutch_Cmd=1, Gen_Load=0	Parallel mode	Engine torque is based on SOC weighting; motor fills remaining torque and grade compensation. Generator torque is zero.
Engine_Cmd=1, Clutch_Cmd=1, Gen_Load=1	Series-parallel mode	Engine holds T_engine_opt. Surplus goes to generator or deficit is supplied by motor.

```

function [T_engine, T_motor, T_gen] = Torque_Split_Dynamic(...
    T_req, SOC, Grade, Clutch_Cmd, Engine_Cmd, Gen_Load, omega_eng)

% CONSTANTS & VEHICLE PARAMETERS
g          = 9.81;          % m/s^2 - Gravity
m_veh      = 1200;          % kg    - Vehicle mass
r_wheel    = 0.3;          % m     - Wheel radius

%% POWERTRAIN PARAMETERS
T_engine_max = 200;          % Nm    - Max engine torque
T_motor_max  = 150;          % Nm    - Max motor torque
T_gen_max    = 150;          % Nm    - Max generator torque capacity
T_engine_opt = 120;          % Nm    - Engine best efficiency torque

eta_gen      = 0.90;          % -    - Generator efficiency
eta_motor    = 0.92;          % -    - Motor efficiency
eta_clutch   = 0.97;          % -    - Clutch mechanical efficiency

%% SOC THRESHOLDS
SOC_low      = 30;          % %    - Low battery threshold
SOC_high     = 80;          % %    - High battery threshold
SOC_target   = 60;          % %    - Ideal SOC target
k_soc        = 0.5;          % -    - SOC error gain

%% PHYSICS CALCULATIONS
theta = atan(Grade / 100);          % [radians]
F_grade = m_veh * g * sin(theta);    % Grade force [N]

W_soc = max(0, min(1, (SOC - SOC_low) / (SOC_high - SOC_low)));
delta_T_charge = k_soc * (SOC_target - SOC);

%% COMMAND-BASED TORQUE SPLIT

% Initialize all values
T_engine = 0; T_motor = 0; T_gen = 0;
omega_engine = omega_eng;

if Engine_Cmd == 0 && Clutch_Cmd == 0 && Gen_Load == 1
    % --- STANDBY or REGEN BRAKING ---
    if T_req < 0
        T_motor = -abs(T_req) * eta_motor;
        T_motor = max(T_motor, -T_motor_max);
    else
        T_motor = 0;
    end
end

elseif Engine_Cmd == 1 && Clutch_Cmd == 0 && Gen_Load == 1
    % --- SERIES MODE ---
    T_motor = min(T_req, T_motor_max);

    if SOC >= SOC_high
        T_engine = 0;
    else
        T_engine = T_engine_opt + delta_T_charge;
        T_engine = max(0, min(T_engine, T_engine_max));
    end

    T_gen = T_engine; % Generator absorbs mechanical torque directly

```

Figure 8.5 Torque_Split_Dynamic code screenshot, part 1.

Listing 8.3 Torque_Split_Dynamic MATLAB Function extracted from the SLX package.

```

function [T_engine, T_motor, T_gen] = Torque_Split_Dynamic(...)

    T_req, SOC, Grade, Clutch_Cmd, Engine_Cmd, Gen_Load, omega_eng)

% CONSTANTS & VEHICLE PARAMETERS

g          = 9.81;          % m/s^2 - Gravity

m_veh      = 1200;          % kg    - Vehicle mass

r_wheel    = 0.3;          % m     - Wheel radius

```

```

%% POWERTRAIN PARAMETERS

T_engine_max = 200;    % Nm - Max engine torque

T_motor_max = 150;    % Nm - Max motor torque

T_gen_max = 150;    % Nm - Max generator torque capacity

T_engine_opt = 120;    % Nm - Engine best efficiency torque


eta_gen = 0.90;    % - - Generator efficiency

eta_motor = 0.92;    % - - Motor efficiency

eta_clutch = 0.97;    % - - Clutch mechanical efficiency


%% SOC THRESHOLDS

SOC_low = 30;    % % - Low battery threshold

SOC_high = 80;    % % - High battery threshold

SOC_target = 60;    % % - Ideal SOC target

k_soc = 0.5;    % - - SOC error gain


%% PHYSICS CALCULATIONS

theta = atan(Grade / 100);    % [radians]

F_grade = m_veh * g * sin(theta);    % Grade force [N]


W_soc = max(0, min(1, (SOC - SOC_low) / (SOC_high - SOC_low)));

delta_T_charge = k_soc * (SOC_target - SOC);


%% COMMAND-BASED TORQUE SPLIT

% Initialize all values

T_engine = 0; T_motor = 0; T_gen = 0;

omega_engine = omega_eng;


if Engine_Cmd == 0 && Clutch_Cmd == 0 && Gen_Load == 1

    % --- STANDBY or REGEN BRAKING ---

    if T_req < 0

        T_motor = -abs(T_req) * eta_motor;

        T_motor = max(T_motor, -T_motor_max);

    else

        T_motor = 0;

    end

end

```

```

elseif Engine_Cmd == 1 && Clutch_Cmd == 0 && Gen_Load == 1

    % --- SERIES MODE ---

    T_motor = min(T_req, T_motor_max);

    if SOC >= SOC_high

        T_engine = 0;

    else

        T_engine = T_engine_opt + delta_T_charge;

        T_engine = max(0, min(T_engine, T_engine_max));

    end

    T_gen = T_engine; % Generator absorbs mechanical torque directly

elseif Engine_Cmd == 1 && Clutch_Cmd == 1 && Gen_Load == 0

    % --- PARALLEL MODE ---

    T_grade_motor = F_grade * r_wheel;

    T_engine = (1 - W_soc) * T_engine_max;

    T_engine = min(T_engine, T_req);

    T_motor = (T_req - (T_engine * eta_clutch)) + T_grade_motor;

    T_motor = max(0, min(T_motor, T_motor_max));

    T_gen = 0;

elseif Engine_Cmd == 1 && Clutch_Cmd == 1 && Gen_Load == 1

    % --- SERIES-PARALLEL MODE ---

    T_engine = T_engine_opt;

    if T_engine >= T_req

        % Engine has surplus. Calculate torque gap and route to generator.

        T_surplus = T_engine - T_req;

        T_gen = T_surplus;

        T_motor = 0;

    else

        % Engine has deficit. Motor must assist.

        T_motor = min(T_req - T_engine, T_motor_max);

        T_gen = 0;

    end

end

```

```

T_engine = max(0, min(T_engine, T_engine_max));

T_motor = max(-T_motor_max, min(T_motor, T_motor_max));

% BUG 4 FIXED: Clamped to T_gen_max instead of T_engine_max
T_gen = max(0, min(T_gen, T_gen_max));

end

elseif Engine_Cmd == 1 && Clutch_Cmd == 1 && Gen_Load == 0
    % --- PARALLEL MODE ---
    T_grade_motor = F_grade * r_wheel;

    T_engine = (1 - W_soc) * T_engine_max;
    T_engine = min(T_engine, T_req);

    T_motor = (T_req - (T_engine * eta_clutch)) + T_grade_motor;
    T_motor = max(0, min(T_motor, T_motor_max));
    T_gen = 0;

elseif Engine_Cmd == 1 && Clutch_Cmd == 1 && Gen_Load == 1
    % --- SERIES-PARALLEL MODE ---
    T_engine = T_engine_opt;

    if T_engine >= T_req
        % Engine has surplus. Calculate torque gap and route to generator.
        T_surplus = T_engine - T_req;
        T_gen = T_surplus;
        T_motor = 0;
    else
        % Engine has deficit. Motor must assist.
        T_motor = min(T_req - T_engine, T_motor_max);
        T_gen = 0;
    end
end

T_engine = max(0, min(T_engine, T_engine_max));
T_motor = max(-T_motor_max, min(T_motor, T_motor_max));

% BUG 4 FIXED: Clamped to T_gen_max instead of T_engine_max
T_gen = max(0, min(T_gen, T_gen_max));

end

```

Figure 8.6 Torque_Split_Dynamic code screenshot, part 2.

CHAPTER 9: SCOPES AND RESULT DOCUMENTATION

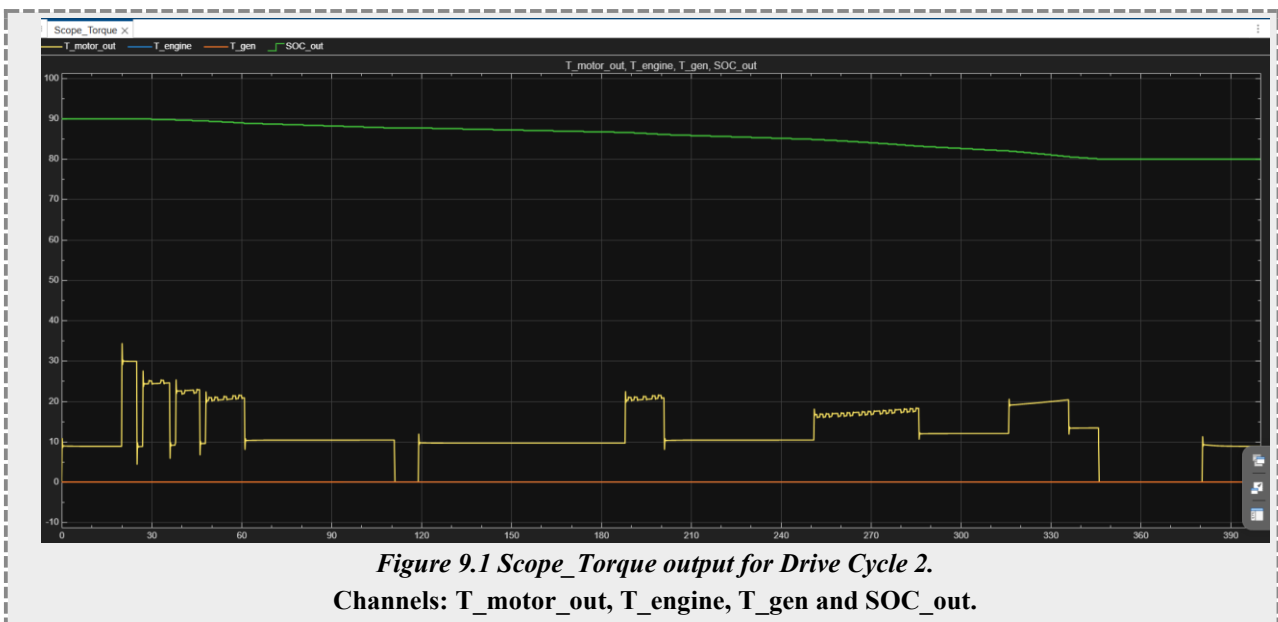
The uploaded model contains several top-level scopes. The scope signal labels were extracted from the Scope GraphicalSettings saved in the SLX XML. These scopes are essential for presenting simulation results in the final report.

Result-screenshot note: all scope plots inserted in this section correspond to Drive Cycle 2. The plots should therefore be interpreted as the model response under the second drive-cycle/terrain selection.

Scope name	No. of inputs	Signals connected	Recommended interpretation result
Scope_Torque	4	T_motor_out, T_engine, T_gen, SOC_out	Shows torque split and SOC trend together. Use it to verify mode decisions and torque allocation.
Scope_Speed	2	Physical Plant/feedback, control/Clutch_Cmd	Shows vehicle speed response and clutch engagement timing.
Scope_Power	3	Physical Plant/P, Motor_control/P_motor_act, Generator_control/P_gen	Shows mechanical/electrical power exchange among engine, motor, and generator.
Scope_GenEng	3	Physical Plant/wEng, T_gen, Generator_control/I_gen	Shows engine speed, generator torque, and charging current.
Scope_battery	3	Battery/SOC, Battery/V_terminal, Generator_control/I_gen	Shows SOC, terminal voltage, and generator charging current.
Scope_Mode	2	control/Clutch_Cmd, control/Gen_Load	Shows clutch engagement command and generator-load command during Drive Cycle 2.

9.1 Scope_Torque expected graph notes

In series mode, motor torque should track propulsion demand while engine/generator torque supports charging. In parallel mode, engine torque should increase and motor torque should assist only as required. In regenerative braking, motor torque may become negative if T_{req} is negative and SOC protection permits regeneration.



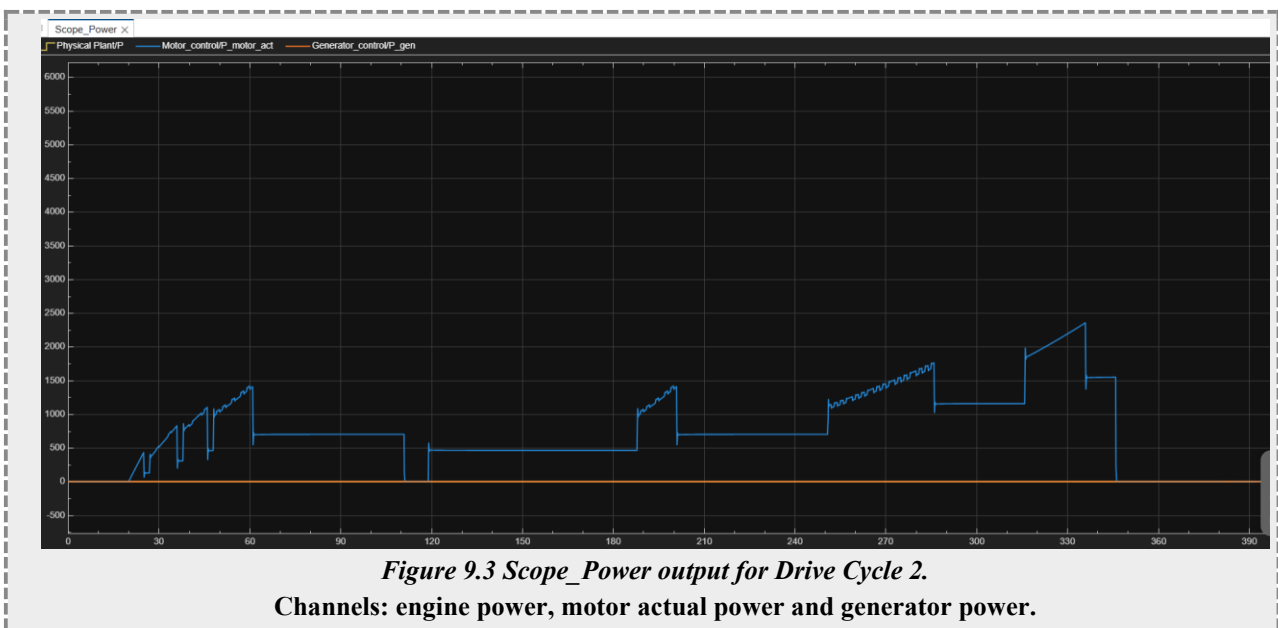
9.2 Scope_Speed expected graph notes

The speed scope should show the actual vehicle-speed feedback. It can be compared against the selected drive-cycle reference inside control. Clutch_Cmd can be overlaid to show how mode transitions affect driveline engagement.



9.3 Scope_Power expected graph notes

The power scope helps confirm energy flow. During propulsion, motor power should generally be positive when the motor assists. During generator charging, Generator_control/P_gen should rise with generator torque and engine speed. Engine power should correspond to engine torque and speed behavior in the plant sensor subsystem.



9.4 Scope_GenEng and Scope_battery expected graph notes

Scope_GenEng is useful for checking generator charging logic. Scope_battery is useful for checking whether SOC and $V_{terminal}$ stay within limits. If SOC rises while I_{gen} is positive and motor power is low, the battery/generator path is behaving as expected. If $V_{terminal}$ clips at the clamp limit, the pack parameters should be reviewed.

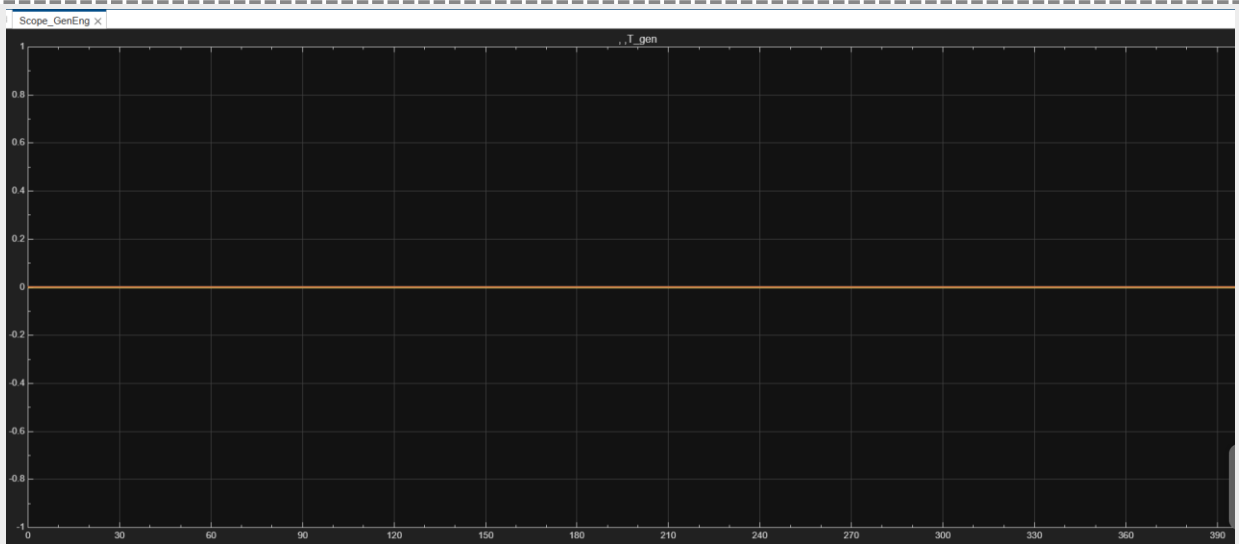


Figure 9.4 Scope_GenEng output for Drive Cycle 2.

Channels: engine speed, generator torque and generator current; the shown run indicates generator torque/current remain nearly zero.



Figure 9.5 Scope_battery output for Drive Cycle 2.

Channels: SOC, V_terminal and I_gen.

9.5 Additional Drive Cycle 2 diagnostic scopes

The following additional scope captures are included because they directly support controller verification for the same Drive Cycle 2 simulation run.

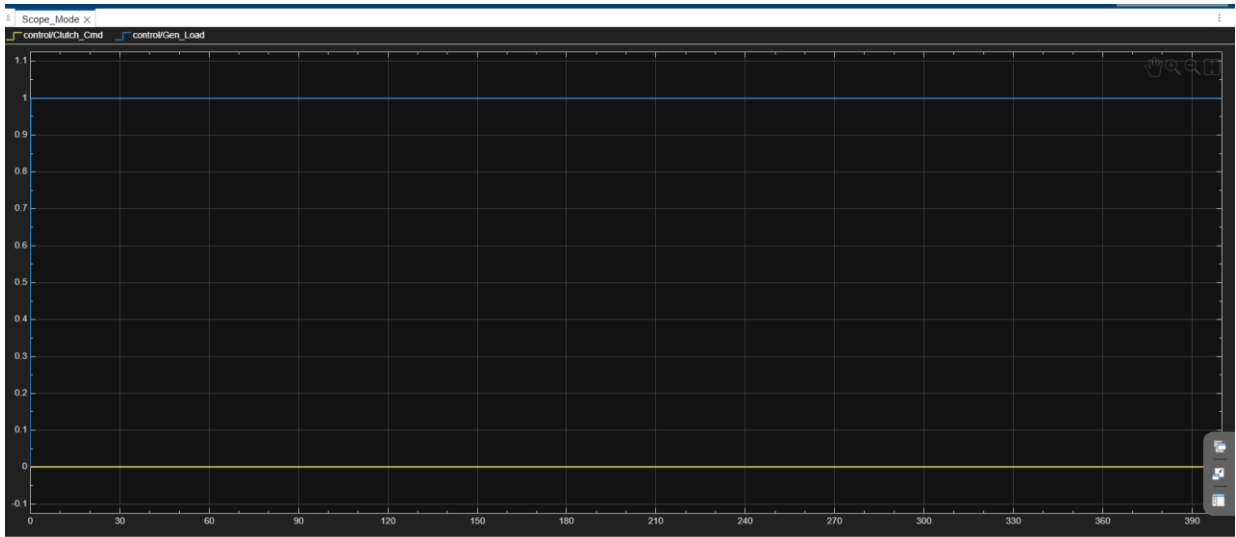


Figure 9.6 Scope_Mode output for Drive Cycle 2: Clutch_Cmd and Gen_Load commands.

The plot shows generator loading enabled and clutch command inactive for the shown drive-cycle run.



Figure 9.7 DriverPI/internal speed-tracking scope for Drive Cycle 2.

This plot compares v_{ref} and V_{actual} and also shows the filtered T_{req} response generated by the driver controller.



Figure 9.8 EMS chart command scope for Drive Cycle 2.

Shows Clutch_Cmd, Engine_Cmd and Gen_Load command levels produced by the mode-selection chart.

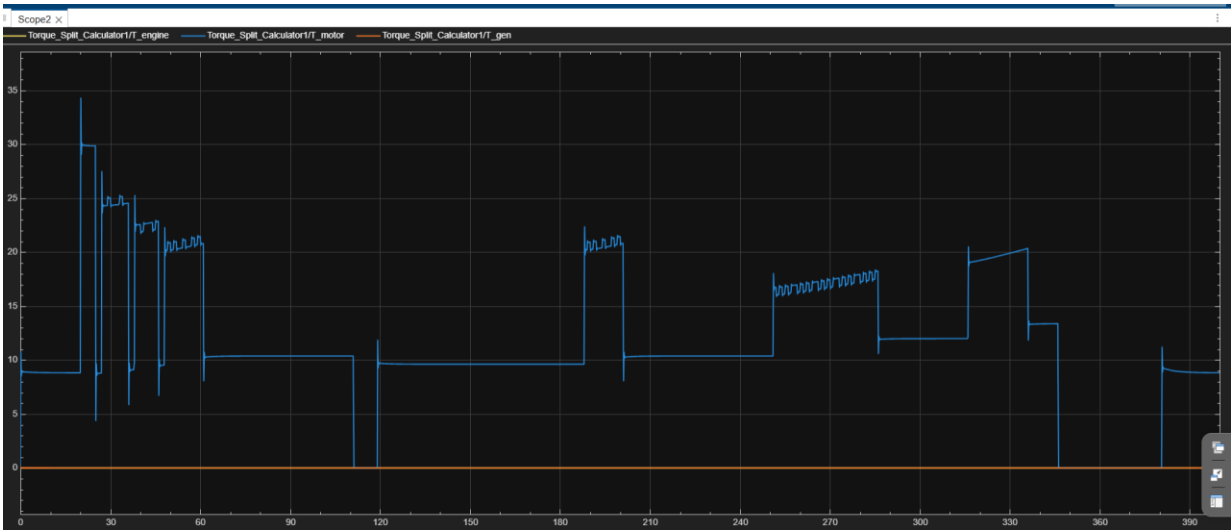


Figure 9.9 Torque split calculator output scope for Drive Cycle 2.

Shows the internal T_engine, T_motor and T_gen torque commands before/around final output routing.

CHAPTER 10: PARAMETER REFERENCE

This table lists tunable constants and key parameters visible in the current uploaded file. Some values are numeric and directly embedded in blocks or code. Others are external workspace symbols and must be defined in the MATLAB base workspace, model workspace, or initialization script before simulation.

Parameter	Current value	Unit	Source in model	Comment
Kp	2	-	control/Constant DriverPI input	-> Proportional speed-error gain used by the current PI driver controller.
DriverPI alpha	0.02	-	control/MATLAB Function script	Low-pass smoothing factor for T_req filter.
DriverPI torque scale	280	Nm	control/MATLAB Function script	T_req_raw = max(0,pedal)*280.
N_series	5	cells in series	Battery/Constant BatteryECM input	-> Pack-level voltage scaling. Clamp range becomes 12.5 V to 21.5 V.
SOC initial condition	SOC_init (external symbol); Drive Cycle 2 run begins near 90%	fraction	Battery/Unit initial condition	Delay Initial SOC is workspace-defined in the current SLX; do not document it as a fixed 0.6 unless SOC_init is set to 0.6.
OCV lookup BP	[0,0.1,...,1]	SOC fraction	Battery/1-D Table	Lookup SOC breakpoints for cell OCV curve.
OCV lookup table	[3,3.2,3.35,3.45,3.55,3.65,3.72,3.78,3.85,3.93,4.2]	V/cell	Battery/1-D Table	Lookup Cell OCV values.
R0 lookup BP	[0,0.2,0.5,0.8,1]	SOC fraction	Battery/1-D Table1	Lookup SOC breakpoints for internal resistance.
R0 lookup table	[0.08,0.06,0.05,0.05,0.06]	ohm/cell	Battery/1-D Table1	Lookup Ohmic resistance values used in pack voltage drop.
C_nom	external symbol	Ah	BatteryECM MATLAB Function parameter	Referenced by code but numeric value is not embedded in the SLX XML/package.
R1	external symbol	ohm	BatteryECM MATLAB Function parameter	Referenced by code but numeric value is not embedded in the SLX XML/package.
C1	external symbol	F	BatteryECM MATLAB Function parameter	Referenced by code but numeric value is not embedded in the SLX XML/package.
I_max_charge	external symbol	A	GeneratorPower MATLAB Function parameter	Current clamp parameter; numeric value not embedded in the SLX XML/package.

T_gen_max	external symbol in Generator_control saturation; 150 in Torque_Split code	Nm	Generator_control/Saturation + Torque_Split_Dynamic	Workspace limit is symbolic in Generator_control, while Torque_Split_Dynamic uses 150 Nm internally.
T_engine_max	200	Nm	Torque_Split_Dynamic code	Maximum engine torque inside torque split calculator.
T_motor_max	150	Nm	Torque_Split_Dynamic code	Maximum motor torque inside torque split calculator.
T_engine_opt	120	Nm	Torque_Split_Dynamic code	Engine operating target used in series and series-parallel modes.
SOC_low	30	%	Torque_Split_Dynamic code	Low SOC threshold.
Ki	0.05	-	control/Constant2 -> DriverPI input	Integral gain used in current DriverPI function.
DriverPI dt	0.01	s	control/Constant3 -> DriverPI input	Time step used for DriverPI error integration.
Battery SOC_init	external symbol	fraction	Battery/Unit Delay initial condition	Workspace-defined initial SOC for discrete BatteryECM feedback path.
Battery V_RC_init	external symbol	V/cell equivalent	Battery/Unit Delay1 initial condition	Workspace-defined initial RC voltage state.
Parameter	Current value	Unit	Source in model	Comment
SOC_high	80	%	Torque_Split_Dynamic code	High SOC threshold.
SOC_target	60	%	Torque_Split_Dynamic code	Target SOC for charging torque correction.
k_soc	0.5	-	Torque_Split_Dynamic code	Gain for SOC-target charging torque correction.
eta_gen	0.90	-	Torque_Split_Dynamic code	Declared generator efficiency; not directly used elsewhere in the shown torque split equations.
eta_motor	0.92	-	Torque_Split_Dynamic code	Used for regenerative motor torque scaling.
eta_clutch	0.97	-	Torque_Split_Dynamic code	Used in parallel-mode torque allocation.

Vehicle mass	1200	kg	Physical Plant Vehicle Body; Torque_Split_Dynamic code	Used in plant and grade force calculation.
Wheel radius	0.3	m	Torque_Split_Dynamic code	Used for grade compensation torque. Tire block separately uses Magic Formula parameters.
Motor Gear ratio	2	-	Physical Plant/Motor Gear1	Simple gear block ratio.
Final Drive ratio	2	-	Physical Plant/Final Drive gear1	Simple gear block ratio.
Gear efficiency parameter	0.8	-	Physical Plant/Motor Gear1 and Final Drive gear1	Simple gear efficiency parameter; friction model is set to off.
Engine shaft inertia	0.25	kg*m ²	Physical Plant/Engine Shaft Inertia	Dedicated engine shaft inertia.
Driveline inertia	0.01	kg*m ²	Physical Plant/Driveline_Inertia1	Driveline inertia block.
Motor torque-source inertia	0.01	kg*m ²	Physical Plant/Inertia T_motor1	Motor shaft inertia.
Generator torque-source inertia	0.01	kg*m ²	Physical Plant/Inertia T_gen1	Generator shaft inertia.
Wheel inertia	0.01	kg*m ²	Physical Plant/Wheel Inertia1	Wheel rotational inertia block.
Vehicle frontal area	3	m ²	Physical Plant/Vehicle Body1	Aerodynamic drag area.
Vehicle Cd	0.4	-	Physical Plant/Vehicle Body1	Aerodynamic drag coefficient.
Parameter	Current value	Unit	Source in model	Comment
Air density	1.18	kg/m ³	Physical Plant/Vehicle Body1	Vehicle body air density parameter.
Tire pressure	250e3	Pa	Physical Plant/Tire (Magic Formula)1	Magic Formula tire pressure.
Tire damping	1000	N*s/m	Physical Plant/Tire (Magic Formula)1	Tire damping parameter.
Clutch effective radius	130	mm	Physical Plant/Disc Friction Clutch1	Clutch effective friction radius.
Clutch piston area	0.001	m ²	Physical Plant/Disc Friction Clutch1	Hydraulic piston area.
Clutch static friction coefficient	0.35	-	Physical Plant/Disc Friction Clutch1	Fixed static coefficient.

Brake gain	-500	-	Physical Plant/Gain	Converts brake signal to physical braking input direction.
Clutch command scaling	1/200 then saturation [0,1]	-	Physical Plant/Gain1 + Saturation	Scales command into clutch actuation domain.
Grade-to-angle	atan(Grade/100)	rad	Physical Plant/MATLAB Function3	Road grade conversion for vehicle body road angle input.

10.1 External workspace parameters requiring confirmation

Symbol	Where used	Why it must be confirmed
C_nom	BatteryECM MATLAB Function	Affects SOC rate directly. Wrong value changes battery depletion/charging speed.
R1, C1	BatteryECM MATLAB Function	Affects RC voltage dynamics and terminal-voltage transient response.
I_max_charge	GeneratorPower MATLAB Function	Limits generator charging current.
T_gen_max	Generator_control Saturation block	Limits actual generator torque.
dt	BatteryECM, DriverPI, and From Workspace sample times	Affects battery state update, PI integration, and drive-cycle sampling.
dc_ts{1}...dc_ts{6}	control drive-cycle From Workspace blocks	Provide speed reference signals for terrain modes.
SOC_init	Battery Unit Delay initial condition	Sets initial SOC; current report should not assume a fixed value unless this workspace variable is known.
V_RC_init	Battery Unit Delay1 initial condition	Sets initial RC network voltage state for the 1RC battery model.

10.2 Parameter initialization and workspace setup

The Simulink model is initialized through a combined MATLAB parameter script. This script must be run before simulation so that the drive cycles, HEV_Param structure, battery ECM variables, generator limits, lookup tables, and DriverPI gains are available in the MATLAB workspace. The complete script listing is provided in Appendix C.

Key initialization groups documented in the uploaded parameter file are:

- Drive cycle setup: loads UrbanCycle1 to UrbanCycle6 and creates dc_ts and v_ref timeseries.
- HEV_Param structure: stores battery, motor, generator, DC-DC converter, controller, vehicle, power-split, and ICE parameters.
- 1RC ECM battery variables: defines C_nom, V_nom, R0, SOC_init, n_series, OCV/R0 lookup tables, R1-C1 branch, and dt.
- Control and generator constants: defines T_gen_max, I_max_charge, generator efficiency map, Kp_driver, and Ki_driver.

CHAPTER 11: LIMITATIONS AND ASSUMPTIONS

Area	Limitation / assumption
External workspace parameters	Several symbols are referenced but not numerically stored in the SLX package: C_nom, R1, C1, I_max_charge, T_gen_max, dt, SOC_init, V_RC_init, and dc_ts{1}...dc_ts{6}.
Generator efficiency application	The latest SLX shows the efficiency lookup output connected to the eta input of GeneratorPower, and the function multiplies by eta. Remaining validation should focus on map calibration and speed-unit consistency.
DriverPI implementation	The latest SLX implements PI behavior with Kp, Ki, dt, anti-windup, and low-pass filtered T_req. The gains still need tuning against the selected drive cycle.
Speed units in power calculation	omega_eng appears labelled as rpm in the model. Mechanical power in physical watts requires angular speed in rad/s; therefore P_gen and engine power should be interpreted carefully unless speed units are confirmed.
Torque limit inconsistency	Torque_Split_Dynamic clamps engine/motor/generator to 200/150/150 Nm, while output Saturation blocks in control use 250/200/200 or similar limits. The final command is affected by both sets of limits.
Simplified road grade	generate_terrain_scenario assigns fixed grade values based on Terrain_Mode: 0%, 1%, or 8%. It does not model continuously varying road slope unless extended.
Battery pack scale	N_series=5 creates a low-voltage pack representation. This is acceptable for control demonstration but not a realistic full HEV pack unless intentionally scaled.
Physical plant simplifications	Ideal torque sources simplify engine, motor, and generator behavior. Thermal effects, detailed combustion maps, inverter dynamics, and battery thermal limits are not fully modeled.
Stateflow hysteresis	The EMS uses thresholds such as SOC 30/40 and speed 70/80, which provide some separation. Additional time hysteresis/debounce could further reduce mode chatter.
Validation scope	The model structure is documented from the uploaded SLX and PDF screenshots. Numerical simulation correctness should still be validated inside MATLAB/Simulink with the required workspace variables loaded.

CHAPTER 12: VERIFICATION AND FUTURE WORK

12.1 Simulation verification checklist

Model audit note: This final pass was checked against HEV_2_7_26_copy.slx. The major corrections made in this document are: control now has 3 top-level inputs, BatteryECM is discrete-time with dt, GeneratorPower uses eta, and DriverPI includes Ki, dt, anti-windup, and filtered T_req.

Check area	What to verify
Workspace setup	Confirm C_nom, R1, C1, I_max_charge, T_gen_max, dt, SOC_init, V_RC_init, and dc_ts{1}...dc_ts{6} exist before simulation.
Interface correctness	Verify the top-level ports match the tables in Chapter 3.
SOC units	Confirm control receives SOC in percent while Battery internally integrates SOC as 0-1 fraction.
Speed feedback	Confirm Physical Plant feedback is in km/h and is routed to control and Motor_control correctly.
Generator current	Check I_gen never exceeds I_max_charge and remains non-negative.
Motor derating	Test SOC below 10-20% and above 80-85% to verify K_SOC and K_regen behavior.
Mode selection	Test city, highway, mountain, braking, high SOC, low SOC, and high grade cases.
Torque split	Check T_engine, T_motor, and T_gen in each Stateflow mode against the logic table.
Plant response	Check vehicle speed changes smoothly and no algebraic loop or solver errors occur.
Scope export	Export scope plots with legends and units for final submission.

12.2 Suggested future improvements

- Verify the generator efficiency map over a wider operating range and confirm whether omega_eng entering GeneratorPower is in rpm or rad/s before interpreting P_gen as physical watts.
- Tune Kp, Ki, dt, and the T_req filter alpha for better drive-cycle tracking and reduced oscillations in torque request.
- Move hardcoded constants from Torque_Split_Dynamic into a single initialization script or model workspace for traceability.
- Use realistic HEV battery pack N_series and capacity values, with clear sourcing.
- Add mode hysteresis with timers to reduce Stateflow transition chatter.
- Add signal logging to Simulation Data Inspector and export standard result plots automatically.
- Validate physical plant against expected acceleration, gradeability, and energy consumption values.

12.3 Conclusion

The uploaded model forms a complete student-level HEV simulation architecture with battery, generator, motor, physical plant, and EMS/control subsystems. The model is especially useful for demonstrating how Stateflow mode logic interacts with torque split, motor limits, generator charging, and vehicle dynamics. The latest model revision has improved implementation consistency by using eta in GeneratorPower and by changing DriverPI from P-only behavior to PI behavior with anti-windup. Remaining improvement areas are parameter traceability, physical

unit consistency, drive-cycle calibration, and validation against expected HEV performance values.

REFERENCES

- [1] MathWorks, Simulink documentation for model-based design, MATLAB Function blocks, and simulation workflows.
- [2] MathWorks, Simscape Driveline documentation for rotational mechanical networks, gears, tires, and vehicle bodies.
- [3] MathWorks, Stateflow documentation for state machines and control logic in Simulink models.
- [4] IEEE, IEEE Conference Template and authoring guidelines for technical papers.

APPENDIX A: EXTRACTED MATLAB FUNCTION CODE

A.1 BatteryECM

```
function [SOC_new, V_terminal, V_RC_new, I_batt_out] = ...
    BatteryECM(I_batt, OCV, R0, SOC_in, V_RC_in, dt, C_nom, R1, C1, N_series)

%% 1. SOC integration (Coulomb counting)
dSOC = -I_batt * dt / (C_nom * 3600);
SOC_new = SOC_in + dSOC;
SOC_new = max(0, min(1, SOC_new));

%% 2. RC pair dynamics at cell level
alpha = exp(-dt/(R1*C1));
V_RC_new = alpha*V_RC_in + R1*(1-alpha)*I_batt;

%% 3. Terminal voltage at pack level
OCV_pack = OCV * N_series;
R0_pack_drop = I_batt * R0 * N_series;
V_RC_pack = V_RC_new * N_series;

V_terminal = OCV_pack - R0_pack_drop - V_RC_pack;
V_terminal = max(2.5*N_series, min(4.3*N_series, V_terminal));

%% 4. Pass-through
I_batt_out = I_batt;
end
```

A.2 GeneratorPower

```
function [P_gen, I_gen] = GeneratorPower(T_gen_sat, omega_eng, V_bat, eta, I_max_charge)

% Electrical power generation
P_gen = T_gen_sat * omega_eng * eta;

% Avoid divide-by-zero
V_nom_min = 1;
V_safe = max(V_bat, V_nom_min);

% Charging current
I_gen = P_gen / V_safe;

% Current limit
I_gen = min(I_gen, I_max_charge);
```

```
% No negative charging current
```

```
I_gen = max(I_gen,0);
```

```
end
```

A.3 DriverPI

```
function [T_req, brake_sig, pedal, err] = DriverPI(v_ref, v_actual, Kp, Ki, dt)
```

```
% 1. Initialize memory
```

```
persistent int_err T_req_filt
```

```
if isempty(int_err)
```

```
    int_err = 0;
```

```
    T_req_filt = 0;
```

```
end
```

```
% 2. Error calculation
```

```
err = v_ref - v_actual;
```

```
% 3. Proper time-based integration
```

```
int_err = int_err + (err * dt);
```

```
% 4. Anti-windup
```

```
if Ki ~= 0
```

```
    int_limit = 1 / abs(Ki);
```

```
    int_err = max(-int_limit, min(int_limit, int_err));
```

```
end
```

```
% 5. PI control law
```

```
pedal_unsat = (Kp * err) + (Ki * int_err);
```

```
pedal = max(-1, min(1, pedal_unsat));
```

```
% 6. Torque and braking commands
```

```
T_req_raw = max(0, pedal) * 280;
```

```
brake_sig = max(0, -pedal);
```

```
% 7. Low-pass filter for T_req
```

```
alpha = 0.02;
```

```
T_req_filt = T_req_filt + alpha * (T_req_raw - T_req_filt);
```

```
T_req = T_req_filt;
```

```
end
```

A.4 generate_terrain_scenario

```
function [T_req, Grade] = generate_terrain_scenario(Terrain_Mode, T_req_in)
```

```
% #codegen
```

```
% This block generates simulated vehicle states based on selected terrain.
```

```
%
```

```
% Input:
```

```
% Terrain_Mode - 1 = City, 2 = Highway, 3 = Mountain, etc.
```

```
%
```

```
% Outputs:
```

```
% T_req - Driver torque request in Nm
```

```
% Grade - Road gradient in %
```

```
switch Terrain_Mode
```

```
    case 1
```

```
        % drive cycle 1
```

```
        T_req = T_req_in; % Nm
```

```
        Grade = 0.0; % flat road
```

```
    case 2
```

```
        % drive cycle 2
```

```
        T_req = T_req_in; % Nm
```

```
        Grade = 1.0; % slight road grade
```

```
    case 3
```

```
        % drive cycle 3
```

```
        T_req = T_req_in; % Nm
```

```
        Grade = 8.0; % steep uphill road
```

```
    case 4
```

```
        % drive cycle 4
```

```
        T_req = T_req_in; % Nm
```

```
        Grade = 0.0; % flat road
```

```
    case 5
```

```
        % drive cycle 5
```

```
        T_req = T_req_in; % Nm
```

```
        Grade = 1.0; % slight road grade
```

```
    case 6
```

```
        % drive cycle 6
```

```
        T_req = T_req_in; % Nm
```

```
        Grade = 8.0; % steep uphill road
```

```
otherwise
```

```
    % DEFAULT / FAILSAFE
```

```
    T_req = 0.0; % Safely zero out torque demand
```

```
    Grade = 0.0; % Assume flat ground
```

```
end
```

```
end
```

A.5 grade_to_angle

```
function beta = grade_to_angle(Grade)

%#codegen

beta = atan(Grade/100);

end
```

A.6 Torque_Split_Dynamic

```
function [T_engine, T_motor, T_gen] = Torque_Split_Dynamic(...

    T_req, SOC, Grade, Clutch_Cmd, Engine_Cmd, Gen_Load, omega_eng)

%% CONSTANTS & VEHICLE PARAMETERS

g      = 9.81;      % m/s^2 - Gravity

m_veh  = 1200;      % kg   - Vehicle mass

r_wheel = 0.3;      % m    - Wheel radius

%% POWERTRAIN PARAMETERS

T_engine_max = 200;      % Nm   - Max engine torque

T_motor_max  = 150;      % Nm   - Max motor torque

T_gen_max    = 150;      % Nm   - Max generator torque capacity

T_engine_opt = 120;      % Nm   - Engine best efficiency torque

eta_gen      = 0.90;      % -    - Generator efficiency

eta_motor    = 0.92;      % -    - Motor efficiency

eta_clutch   = 0.97;      % -    - Clutch mechanical efficiency

%% SOC THRESHOLDS

SOC_low      = 30;        % %    - Low battery threshold

SOC_high     = 80;        % %    - High battery threshold

SOC_target   = 60;        % %    - Ideal SOC target

k_soc        = 0.5;       % -    - SOC error gain

%% PHYSICS CALCULATIONS

theta = atan(Grade / 100);      % [radians]

F_grade = m_veh * g * sin(theta);      % Grade force [N]

W_soc = max(0, min(1, (SOC - SOC_low) / (SOC_high - SOC_low)));

delta_T_charge = k_soc * (SOC_target - SOC);
```



```

%% COMMAND-BASED TORQUE SPLIT

% Initialize all values
T_engine = 0; T_motor = 0; T_gen = 0;
omega_engine = omega_eng;

if Engine_Cmd == 0 && Clutch_Cmd == 0 && Gen_Load == 1
    % --- STANDBY or REGEN BRAKING ---
    if T_req < 0
        T_motor = -abs(T_req) * eta_motor;
        T_motor = max(T_motor, -T_motor_max);
    else
        T_motor = 0;
    end

elseif Engine_Cmd == 1 && Clutch_Cmd == 0 && Gen_Load == 1
    % --- SERIES MODE ---
    T_motor = min(T_req, T_motor_max);

    if SOC >= SOC_high
        T_engine = 0;
    else
        T_engine = T_engine_opt + delta_T_charge;
        T_engine = max(0, min(T_engine, T_engine_max));
    end

    T_gen = T_engine; % Generator absorbs mechanical torque directly

elseif Engine_Cmd == 1 && Clutch_Cmd == 1 && Gen_Load == 0
    % --- PARALLEL MODE ---
    T_grade_motor = F_grade * r_wheel;

    T_engine = (1 - W_soc) * T_engine_max;
    T_engine = min(T_engine, T_req);

    T_motor = (T_req - (T_engine * eta_clutch)) + T_grade_motor;
    T_motor = max(0, min(T_motor, T_motor_max));
    T_gen = 0;

elseif Engine_Cmd == 1 && Clutch_Cmd == 1 && Gen_Load == 1
    % --- SERIES-PARALLEL MODE ---

```

```

T_engine = T_engine_opt;

if T_engine >= T_req

    % Engine has surplus. Calculate torque gap and route to generator.

    T_surplus = T_engine - T_req;

    T_gen    = T_surplus;

    T_motor  = 0;

else

    % Engine has deficit. Motor must assist.

    T_motor = min(T_req - T_engine, T_motor_max);

    T_gen    = 0;

end

end

T_engine = max(0, min(T_engine, T_engine_max));

T_motor  = max(-T_motor_max, min(T_motor, T_motor_max));

% BUG 4 FIXED: Clamped to T_gen_max instead of T_engine_max

T_gen    = max(0, min(T_gen, T_gen_max));

end

```

APPENDIX B: MODEL INVENTORY TABLES

The following inventories were extracted from the SLX XML files. They are included so that future users do not have to reverse-engineer block names from the Simulink model.

B.1 Top-level model inventory

SID	Block type	Block name
112	SubSystem	Battery
233	Gain	Gain
185	SubSystem	Generator_control
140	SubSystem	Motor_control
18	SubSystem	Physical Plant
242	Scope	Scope_GenEng
241	Scope	Scope_Mode
229	Scope	Scope_Power
227	Scope	Scope_Speed
228	Scope	Scope_Torque
240	Scope	Scope_battery
418	SubSystem	control

B.2 Battery subsystem inventory

SID	Block type	Block name
169	Inport	P_motor_act
124	Inport	I_gen
125	Lookup_n-D	1-D Lookup Table
126	Lookup_n-D	1-D Lookup Table1
386	Constant	Constant
121	Product	Divide
115	SubSystem	MATLAB Function
128	Scope	Scope
127	Sum	Sum
116	UnitDelay	Unit Delay
117	UnitDelay	Unit Delay1
161	UnitDelay	Unit Delay3
129	Outport	SOC
132	Outport	V_terminal

B.3 Generator_control subsystem inventory

SID	Block type	Block name
187	Inport	Gen_Load
186	Inport	T_gen

188	Inport	omega_eng
189	Inport	V_bat
74	Lookup_n-D	2-D Lookup Table
75	SubSystem	MATLAB Function
70	Product	Product
73	Saturate	Saturation
190	Outport	T_gen_out
191	Outport	I_gen
231	Outport	P_gen

B.4 Motor_control subsystem inventory

SID	Block type	Block name
145	Inport	feedback
144	Inport	SOC
143	Inport	T_motor
147	Lookup_n-D	1-D Lookup Table
148	Lookup_n-D	1-D Lookup Table1
150	Lookup_n-D	1-D Lookup Table2
152	Constant	Constant
245	Gain	Gain
149	Product	Product
151	Product	Product1
156	Product	Product2
153	RateLimiter	Rate Limiter
154	Reference	Saturation Dynamic
246	UnitDelay	Unit Delay
160	Outport	P_motor_act
158	Outport	T_motor_out

B.5 Physical Plant inventory

SID	Block type	Block name
54	Inport	T_motor
56	Inport	Grade
55	Inport	T_gen
255	Inport	brake_sig
387	Inport	Clutch_cmd
388	Inport	T_engine
21	Constant	Constant1
474	Reference	Damper
390	Reference	Disc Friction Clutch1

23	Reference	Driveline_Inertia1
475	Reference	Engine Shaft Inertia
25	Reference	Final Drive gear1
256	Gain	Gain
392	Gain	Gain2
26	Gain	Gain3
393	Gain	Gain4
27	Reference	Gen Torque Source2
476	Reference	Generic Engine
472	Goto	Goto
252	Reference	Ideal Rotational Motion Sensor
261	Reference	Ideal Torque Source
29	Reference	Inertia T_gen1
30	Reference	Inertia T_motor1
31	SubSystem	MATLAB Function3
477	Reference	MRRef Engine Housing
260	Reference	Mechanical Rotational Reference
263	Reference	Mechanical Rotational Reference1
33	Reference	Mechanical Rotational Reference4
34	Reference	Mechanical Rotational Reference5
35	Reference	Motor Gear1
36	Reference	Motor Torque Source1
66	Reference	PS Terminator
37	Reference	PS Terminator1
253	Reference	PS-Simulink Converter
39	Reference	PS-Simulink Converter3
254	Product	Product
396	Saturate	Saturation
49	Scope	Scope
478	SubSystem	Sensor
258	Reference	Simulink-PS Converter
414	Reference	Simulink-PS Converter1
40	Reference	Simulink-PS Converter10
41	Reference	Simulink-PS Converter11
42	Reference	Simulink-PS Converter6
43	Reference	Simulink-PS Converter7
415	Reference	Simulink-PS Converter9
45	Reference	Solver Configuration1
46	Reference	Tire (Magic Formula)1
47	Reference	Vehicle Body1
48	Reference	Wheel Inertia1
57	Outport	feedback
417	Outport	engine_power
495	Outport	m

Inventory continued on next page.

SID	Block type	Block name
31::1	Inport	Grade
31::32	Demux	Demux
31::31	S-Function	SFunction
31::33	Terminator	Terminator
31::5	Outport	beta

Inventory continued on next page.

SID	Block type	Block name
480	Goto	Goto3
481	Reference	Ideal Rotational Motion Sensor
482	Reference	Ideal Torque Sensor
483	Reference	Mechanical Rotational Reference
484	Reference	PS-Simulink Converter
485	Reference	PS-Simulink Converter1
486	Product	Product
487	Product	Product1
488	Reference	Transfer Fcn (with initial outputs)
489	Constant	rad/s to rpm
490	PMIOPort	Engine
491	PMIOPort	Trans
492	Outport	m

B.6 control subsystem inventory

SID	Block type	Block name
419	Inport	feedback_v_veh
420	Inport	SOC_in
421	Inport	omega_eng
422	Lookup_n-D	2-D Lookup Table3
423	SubSystem	Chart1
424	Constant	Constant
425	Constant	Constant1
496	Constant	Constant2
497	Constant	Constant3
426	From	From
427	FromWorkspace	From Workspace
428	FromWorkspace	From Workspace1
429	FromWorkspace	From Workspace2
430	FromWorkspace	From Workspace3
431	FromWorkspace	From Workspace4
432	FromWorkspace	From Workspace5
433	Gain	Gain1

434	Goto	Goto
435	Goto	Goto1
436	SubSystem	MATLAB Function
437	SubSystem	MATLAB Function2
438	ManualSwitch	Manual Switch
439	Memory	Memory
440	MultiPortSwitch	Multiport Switch
441	Saturate	Saturation T_engine1
442	Saturate	Saturation T_gen1
443	Saturate	Saturation T_motor1
444	Scope	Scope
445	Scope	Scope1
446	Scope	Scope2
447	SubSystem	Torque_Split_Calculator1
448	SubSystem	Vehicle Speed Controller
464	Outport	SOC_out
465	Outport	T_engine
466	Outport	T_motor
467	Outport	grade
468	Outport	clutch_cmd
469	Outport	Gen_Load
470	Outport	T_gen
471	Outport	brake_sig

Inventory continued on next page.

SID	Block type	Block name
449	Inport	Ref Spd
450	Inport	Veh Spd
451	TransferFcn	20Hz Lowpass Filter
452	Scope	Accel Ctrl
453	Gain	Gain
454	Goto	Goto
455	Gain	Ki
456	Gain	Kp
457	Saturate	Limit Acc
458	Saturate	Limit Brk
459	Integrator	Limits [-5,5]
460	Mux	Mux
461	Sum	Sum
462	Sum	Sum1
463	Outport	Acc

APPENDIX C: PARAMETER INITIALIZATION SCRIPT

This appendix documents the MATLAB initialization script used to configure the model workspace before running the HEV simulation. It combines the drive-cycle loading workflow, the legacy HEV_Param structure, and the flat 1RC ECM battery/generator parameters used by the simplified battery and generator subsystems.

The script should be executed once before simulation. Without this initialization step, workspace-dependent blocks such as the drive-cycle selector, battery ECM, generator controller, motor limits, and PI speed controller may not receive the required constants, lookup tables, or timeseries variables.

C.1 Important parameter groups

- Drive cycles: UrbanCycle1 to UrbanCycle6 are loaded and converted to timeseries form for v_ref selection.
- Driveline and plant parameters: the HEV_Param structure stores motor, generator, vehicle, ICE, DC-DC, battery, power-split, and control data.
- Battery ECM parameters: flat variables such as C_nom, V_nom, R0, SOC_init, n_series, OCV_table, R0_table, R1, C1, tau1, and dt define the simplified 1RC pack model.
- Control parameters: Kp_driver and Ki_driver tune the DriverPI block, while T_gen_max and I_max_charge limit generator torque and charging current.

C.2 Full MATLAB parameter initialization code

The following listing is copied into the report so that the complete simulation initialization can be reproduced without separately opening the MATLAB script file.

```
% Copyright 2011-2026 The MathWorks, Inc.
%% =====
% HEV COMBINED PARAMETERS FILE
% Merges: HEV_Model_PARAM.m (driveline / HEV_Param struct model)
%         battery_init.m      (1RC ECM battery + generator + drive cycle)
% =====

%% =====
% DRIVE CYCLE LOADING (UrbanCycle1-6)
% =====
% Loaded once here and reused by both the DriveCycles array (legacy
% HEV_Param workflow) and the dc_ts / v_ref timeseries (ECM workflow),
% instead of loading each .mat file twice.

load UrbanCycle1
load UrbanCycle2
load UrbanCycle3
load UrbanCycle4
load UrbanCycle5
load UrbanCycle6

DriveCycles(1) = UrbanCycle1;
DriveCycles(2) = UrbanCycle2;
DriveCycles(3) = UrbanCycle3;
DriveCycles(4) = UrbanCycle4;
DriveCycles(5) = UrbanCycle5;
DriveCycles(6) = UrbanCycle6;

Drive_Cycle_Num = 1;

% Build a timeseries for each segment (speed converted km/h -> m/s)
dc_ts = cell(1,6);
for i = 1:6
    seg      = DriveCycles(i);
    t_seg    = seg.time(:);
    v_seg    = seg.Spdkph(:) / 3.6;    % km/h -> m/s

    dc_ts{i} = timeseries(v_seg, t_seg);
```



```

dc_ts{i}.Name          = sprintf('UrbanCycle%d', i);
dc_ts{i}.TimeInfo.Units = 'seconds';

fprintf('Loaded UrbanCycle%d - %d points, max %.1f km/h\n', ...
        i, length(t_seg), max(seg.SpdKph));
end

% Selector (change this number 1-6 to pick the cycle)
selected_cycle = Drive_Cycle_Num;

t_seg = dc_ts{selected_cycle}.Time;
v_seg = dc_ts{selected_cycle}.Data;      % already in m/s

% v_ref timeseries (what a "From Workspace" block reads)
v_ref = timeseries(v_seg, t_seg);
v_ref.Name = 'v_ref';
v_ref.TimeInfo.Units = 'seconds';

% Simulation stop time matched to the selected cycle
t_sim_stop = t_seg(end);

fprintf('Active cycle: UrbanCycle%d | Duration: %.0f s | Max: %.1f km/h\n', ...
        selected_cycle, t_sim_stop, max(v_seg * 3.6));

%% =====
% HEV SERIES-PARALLEL DRIVELINE PARAMETERS (HEV_Param struct)
% =====

% PREDEFINED LI-ION BATTERY PARAMS
HEV_Param.Battery_Det.Nominal_Voltage = 200;
HEV_Param.Battery_Det.Rated_Capacity  = 8.1;      % Ampere-hours
HEV_Param.Battery_Det.Initial_SOC     = 75.75;    % Percent
HEV_Param.Battery_Det.Series_Resistance = 0.2/10;  % Ohm

% FOR ACCEL TEST
%HEV_Param.Battery_Det.Initial_SOC = 97;          % Percent

% GENERIC BATTERY PARAMS
HEV_Param.Battery_Sys.Nominal_Voltage = 217;
HEV_Param.Battery_Sys.Internal_Resistance = 0.24691; % Ohm
HEV_Param.Battery_Sys.Rated_Capacity  = 6.9;      % Ampere-hours
HEV_Param.Battery_Sys.Initial_Charge  = HEV_Param.Battery_Sys.Rated_Capacity*0.9; % Ampere-hours
HEV_Param.Battery_Sys.Expn_Voltage    = 215.0342; % V
HEV_Param.Battery_Sys.Expn_Charge     = 2.3438;

HEV_Param.Battery_Sys.C1.Capacitance  = 2500;
HEV_Param.Battery_Sys.C1.Initial_Voltage = 19;
HEV_Param.Battery_Sys.C1.Series_Resistance = 1e-6;
HEV_Param.Battery_Sys.R2 = 0.3;
HEV_Param.Battery_Sys.R1 = 1.8;
HEV_Param.Battery_Sys.Maximum_Capacity = HEV_Param.Battery_Sys.Rated_Capacity; % Ampere-hours

% ULTRACAPACITOR PARAMETERS
HEV_Param.UltraCapacitor.Nominal_Capacitance = 1000; % Farad
HEV_Param.UltraCapacitor.Rate_C_V           = 0.2;  % Farad/Volt
HEV_Param.UltraCapacitor.Series_R            = 30/3; % Ohm
HEV_Param.UltraCapacitor.Self_Discharge_R    = 500;  % Ohm
HEV_Param.UltraCapacitor.Initial_Voltage     = 217;  % Volt

% MOTOR PARAMETERS
HEV_Param.Motor.Stator_Resistance = 0.0065*14;
HEV_Param.Motor.TorqSpdLUT.SpeedRPM = [0 1200 2000 3000 4000 5000 6000 6500 10000];
HEV_Param.Motor.TorqSpdLUT.TorqueNm = [400 400 225 150 100 80 70 0 0];
HEV_Param.Motor.Damping = 1e-5; % N*m/(rad/s)
HEV_Param.Motor.TorqueControl_TimeConst = 0.02*2/1.5;
HEV_Param.Motor.Shift_Inertia = 0.2;
HEV_Param.Motor.Series_Resistance = 0.01; % CHG
HEV_Param.Motor.Inductances = [0.001597972349731 0.002057052250467];
HEV_Param.Motor.Efficiency = 91;

% GENERATOR PARAMETERS (Simscape driveline generator)
HEV_Param.Generator.Stator_Resistance = 0.05*0.095;
HEV_Param.Generator.Inductances = [0.000635 0.000635];
HEV_Param.Generator.TorqSpdLUT.SpeedRPM = [0 1200 2000 3000 4000 10000 15000];
HEV_Param.Generator.TorqSpdLUT.TorqueNm = [400 400 250 150 110 0 0];
HEV_Param.Generator.Damping = 1e-5; % N*m/(rad/s)
HEV_Param.Generator.TorqueControl_TimeConst = 0.02*2;
HEV_Param.Generator.Shift_Inertia = 0.2;
HEV_Param.Generator.Series_Resistance = 0.01; % CHG

% -- trq indep losses adds compliance -- 10W
HEV_Param.Generator.TorqIndep_Elec_Loss = 0; % W

% CHG -- ADDED COMPLIANCE

```

```

HEV_Param.Generator.Shaft_Stiffness = 1e3;           % W
HEV_Param.Generator.Shaft_Damping   = 1e2;           % W

% DC-DC CONVERTER PARAMETERS
HEV_Param.DCConv.Output_Voltage      = 500;           % Volts
HEV_Param.DCConv.Resistance_Losses   = 1000/40^2;      % Ohm
HEV_Param.DCConv.Kp = 0.01;
HEV_Param.DCConv.Ki = 10;
HEV_Param.DCConv.MinVin = 20;

HEV_Param.DCConv.Mean_Boost.Kp = 0.001;
HEV_Param.DCConv.Mean_Boost.Ki = 1;

HEV_Param.DCConv.EPower2Heat = 0.1;                 % Watts/Watts
HEV_Param.DCConv.Thermal_Mass = 0.1*10;              % kg
HEV_Param.DCConv.Specific_Heat = 100;                % J/kg/K
HEV_Param.DCConv.Initial_Temperature = 25;           % C
HEV_Param.DCConv.Air_Temperature = 298;              % K
HEV_Param.DCConv.Convection.Area = 20;               % cm^2
HEV_Param.DCConv.Convection.HT_Coefficient = 100;     % W/(m^2*K)

% CONTROLLER PARAMETERS
HEV_Param.Control.Engine_Start_RPM = 800;            % RPM
HEV_Param.Control.Engine_Stop_RPM  = 790;            % RPM
HEV_Param.Control.Mode_Logic_TS = 0.1;
HEV_Param.Control.ICE.Kp = 0.02;
HEV_Param.Control.ICE.Ki = 0.01;
HEV_Param.Control.Gen.Kp = 10;
HEV_Param.Control.Gen.Ki = 3;
HEV_Param.Control.Mot.Kp = 500;
HEV_Param.Control.Mot.Ki = 300;
HEV_Param.Control.Veh_Spd.Kp = 0.02;
HEV_Param.Control.Veh_Spd.Ki = 0.04;

% VEHICLE PARAMETERS
HEV_Param.Vehicle.Mass = 600*2;                      % kg
HEV_Param.Vehicle.Tire_Radius = 0.3;                  % m
HEV_Param.Vehicle.Wheel_Inertia = 0.1;                % kg*m^2
HEV_Param.Vehicle.Aero_Drag_Coeff = 0.26;
HEV_Param.Vehicle.Incline = 0;
HEV_Param.Vehicle.Engine_Vehicle_Gear_Ratio = 1.3;
HEV_Param.Vehicle.Distance.CG_FrontAxle = 1.35;
HEV_Param.Vehicle.Distance.CG_RearAxle = 1.35;
HEV_Param.Vehicle.Distance.CG_Ground = 0.5;
HEV_Param.Vehicle.Frontal_Area = 2.16;
HEV_Param.Vehicle.Tire.Rated_Vertical_Load = 3000;    % N
HEV_Param.Vehicle.Tire.Rated_Peak_Long_Force = 3500;  % N
HEV_Param.Vehicle.Tire.Slip_At_Peak_Force = 6;        % Percent
%HEV_Param.Vehicle.Tire.Relaxation_Length = 0.2*0.09; % Percent
HEV_Param.Vehicle.Tire.Relaxation_Length = 0.2;
HEV_Param.Vehicle.Tire.Relaxation_Length = 0.25;
HEV_Param.Vehicle.Trans.Inertia = 0.5;
HEV_Param.Vehicle.Trans.Friction = 0.1*0.001;

% POWER SPLIT DEVICE PARAMETERS
HEV_Param.Power_Split.Ratio_Ring2Sun = 2.6;

% INTERNAL COMBUSTION ENGINE (ICE) PARAMETERS
HEV_Param.ICE.Shaft_Inertia = 0.25;
HEV_Param.ICE.Max_Power = 57000+57000;                % W
HEV_Param.ICE.Speed_At_Max_Power = 5000;              % RPM
HEV_Param.ICE.Max_Speed = 6000;                      % RPM
HEV_Param.ICE.Friction = 0.2079;                     % N*m*s/rad
HEV_Param.ICE.sensor_time_constant = 0.005;          % 0.001

%load FuelConsMap
%HEV_Param.ICE.FC.Speed_Vector = FCMAP.Speed_Vector;
%HEV_Param.ICE.FC.Torque_Vector = FCMAP.Torque_Vector;
%HEV_Param.ICE.FC.Consumption_Table = FCMAP.Consumption_Table;
%clear FCMAP

% BATTERY CELL ELECTROCHEMICAL MODEL - 50Ah (used by the Simscape
% "Battery Cell" block; distinct from the IRC ECM model below, which
% belongs to a different battery representation in this project)
HEV_Param.Battery_Cell.NominalCap = 50;              % (Ah)

HEV_Param.Battery_Cell.SOC_init = 0.8;
HEV_Param.Battery_Cell.theta_init = 25;

HEV_Param.Battery_Cell.Ctheta = 400;                 % (J/degC) Thermal Capacitance
HEV_Param.Battery_Cell.Area = 0.01;                  % (m^2) Surface area exposed to air
HEV_Param.Battery_Cell.Rtheta = 20;                  % (W/m^2/K) Convective heat transfer coeff.

HEV_Param.Battery_Cell.Kc = 1.2;
HEV_Param.Battery_Cell.Costar = 1.8e+005;            % (As)
HEV_Param.Battery_Cell.Kt_Temps = [25 40 60 75];    % Temperature breakpoints for Kt LUT
HEV_Param.Battery_Cell.Kt = {0.80,1.10,1.20,1.12;}; % LUT output values

```

```

HEV_Param.Battery_Cell.delta = 0.73;
HEV_Param.Battery_Cell.Istar = 15;
HEV_Param.Battery_Cell.theta_f = -40;

% (A) Nominal Current (=cap/discharge)
% (degC) Electrolyte Freezing Temp

HEV_Param.Battery_Cell.Ep = 1.95;
HEV_Param.Battery_Cell.Gpo = 2.0e-011;
HEV_Param.Battery_Cell.Vpo = 0.12;
HEV_Param.Battery_Cell.Ap = 2.0;
HEV_Param.Battery_Cell.Taup = 3;

% (V) Parasitic emf
% (s)
% (V)
% (s)

HEV_Param.Battery_Cell.Emo = 2.18*10;
HEV_Param.Battery_Cell.Ke = 0.0006;
HEV_Param.Battery_Cell.Ao = -0.6;
HEV_Param.Battery_Cell.Roo = 0.0042;
HEV_Param.Battery_Cell.R10 = 0.0010;
HEV_Param.Battery_Cell.A21 = -10.0;
HEV_Param.Battery_Cell.A22 = -8.75;
HEV_Param.Battery_Cell.R20 = 0.11;
HEV_Param.Battery_Cell.Taul = 100;

% (V) [max o.c. volts per cell]
% (V/degC)
% (Ohm)
% (Ohm)
% (Ohm)
% (s)

HEV_Param.Battery_Cell.ns = 6;

% Number of cells in series per branch

% Compute initial extracted charge
HEV_Param.Battery_Cell.Qe_init = (1-HEV_Param.Battery_Cell.SOC_init) * ...
    HEV_Param.Battery_Cell.Kc * HEV_Param.Battery_Cell.Costar * ...
    interp1([HEV_Param.Battery_Cell.theta_f HEV_Param.Battery_Cell.Kt_Temps], ...
        [0 HEV_Param.Battery_Cell.Kt], HEV_Param.Battery_Cell.theta_init, 'spline');

%% =====
% 1RC ECM BATTERY MODEL PARAMETERS (from battery_init.m)
% Standalone equivalent-circuit-model battery representation used by a
% separate battery/generator subsystem (flat variables, not part of
% HEV_Param, matching the original script's naming).
% =====

C_nom = 6.5;          % Ah
V_nom = 3.6;          % V
R0 = 0.05;            % Ohm

SOC_high = 0.80;
SOC_low = 0.50;
SOC_min = 0.30;

I_max = 150;          % A

% NOTE: the source battery_init.m set SOC_init twice (0.4, then
% overwritten to 0.90 further down). Only one value can be active --
% pick whichever is correct for your scenario.
SOC_init = 0.90;      % Starting SOC
V_RC_init = 0;        % RC branch voltage starts at 0

% BATTERY PACK PARAMETERS
n_series = 96;         % Example pack
V_pack_nom = V_nom * n_series; % 345.6 V
V_min = 250;           % Minimum battery voltage

% GENERATOR PARAMETERS (ECM-side generator, distinct from
% HEV_Param.Generator above)
T_gen_max = 150;       % Nm
I_max_charge = 45;     % A (semicolon added -- was missing in source)

omega_gen_bp = [0 50 100 150 200 250 300];
eta_gen_table = [0.65 0.82 0.90 0.91 0.90 0.88 0.84];

% OCV / R0 lookup tables vs SOC
OCV_SOC_bp = [0.0 0.1 0.2 0.3 0.4 0.5 ...
    0.6 0.7 0.8 0.9 1.0];

OCV_table = [3.00 3.20 3.35 3.45 3.55 3.65 ...
    3.72 3.78 3.85 3.93 4.20];

R0_SOC_bp = [0.0 0.2 0.5 0.8 1.0];
R0_table = [0.08 0.06 0.05 0.05 0.06];

R1 = 0.02;
C1 = 1500;
taul = R1 * C1;

% SIMULATION PARAMETERS
dt = 0.001;

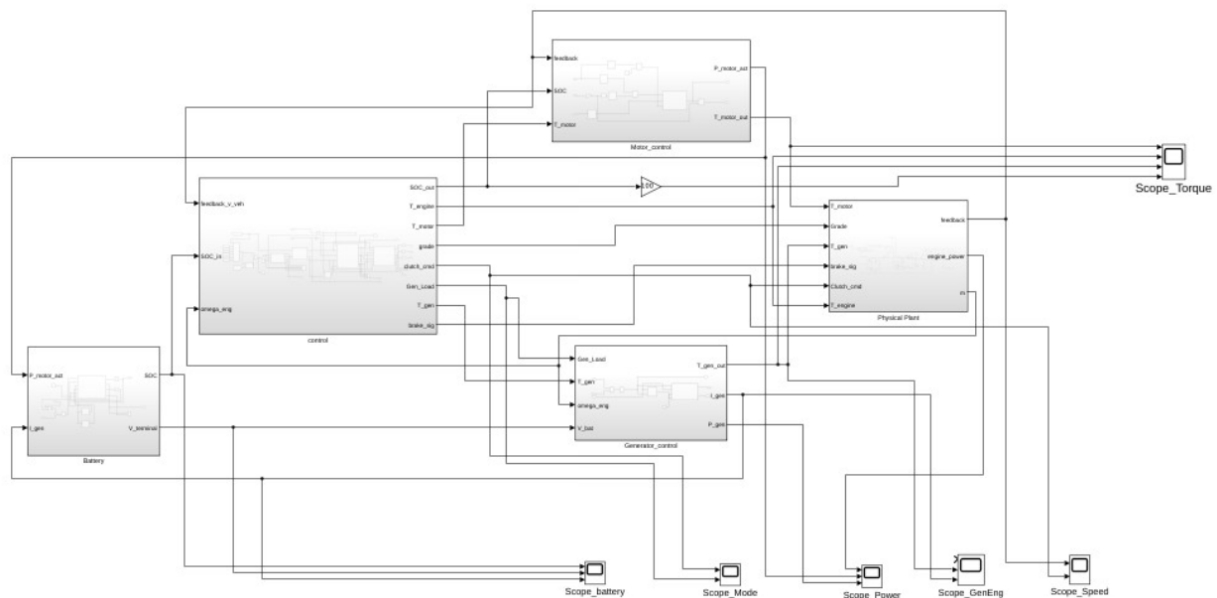
% Driver PI gains
Kp_driver = 0.8;
Ki_driver = 0.05;

```

```
disp('HEV_Combined_Params.m - all parameters loaded (HEV_Param struct, 1RC ECM, drive cycle).');
```

APPENDIX D: SCREENSHOT INDEX FROM UPLOADED PDF

Source	Content used in this report
PDF page 1	Top-level architecture
PDF page 2	Battery subsystem
PDF page 3	BatteryECM code
PDF page 4	Generator_control subsystem
PDF page 5	GeneratorPower code
PDF page 6	Motor_control subsystem
PDF page 7	Physical Plant subsystem
PDF page 8	grade_to_angle code
PDF page 9	Engine sensor subsystem
PDF page 10	control subsystem
PDF page 11	Stateflow EMS chart
PDF page 12	DriverPI code
PDF page 13	generate_terrain_scenario code
PDF page 14	Torque_Split_Dynamic code part 1
PDF page 15	Torque_Split_Dynamic code part 2



Final top-level HEV model screenshot used for submission.

This final image shows the model state after the provided screenshot set and documents the top-level subsystem wiring and scopes.